

Design Document

Students&Companies

Authors:
Luca Ilardi
Matteo Leonardo Luraghi

Politecnico di Milano
Software Engineering II

Academic Year: 2024-25

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms, Abbreviations	3
1.3.1	Definitions	3
1.3.2	Acronyms	3
1.3.3	Abbreviations	3
1.4	Revision history	3
1.5	Reference Documents	3
1.6	Document Structure	4
2	Architectural Design	5
2.1	Overview: High-level components and their interaction	5
2.2	Component view	6
2.2.1	Component diagram	6
2.2.2	Class diagram	7
2.3	Deployment view	7
2.4	Runtime view	8
2.5	Component interfaces	17
2.6	Selected architectural styles and patterns	19
2.6.1	3-tier architecture	19
2.6.2	Model View Controller pattern	19
2.7	Other design decisions	19
2.7.1	Availability	19
2.7.2	Security	19
3	User Interface Design	20
4	Requirements Traceability	30

5	Implementation, Integration and Test Plan	33
5.1	Overview	33
5.2	Implementation plan	33
5.3	Components Integration and Testing	34
5.4	System testing	40
5.5	Additional Specifications on Testing	41
6	Effort Spent	42

Chapter 1

Introduction

1.1 Purpose

As the demand for skilled professionals continues to grow, connecting university students with real-world opportunities has become increasingly important. Traditionally, students search for internships through fragmented processes, often relying on personal networks or generalized job boards, which may not adequately match their skills and aspirations with the needs of companies. This inefficiency results in missed opportunities for both students and employers.

Students&Companies (S&C) is a platform designed to address this challenge by streamlining the internship matchmaking process. It bridges the gap between students seeking internships and companies offering them. S&C allows students to showcase their experiences and skills through CVs while enabling companies to describe their projects and requirements in depth.

Additionally, S&C supports the selection process by facilitating interviews and feedback collection. With features for proactive searching and personalized suggestions, the platform aims to create a seamless and transparent experience for all parties involved.

1.2 Scope

As the RASD describes, two main users interact with the system: Students and Companies.

Students are individuals seeking internship opportunities. They can create and manage their profiles by uploading their CVs. The platform will notify students of relevant internship opportunities based on their profile information and allow them to apply directly. In addition, students can track their applications' status and receive company feedback. The system also provides a Recommendation Engine to suggest internships tailored to students' qualifications and preferences. Students can actively participate by proactively searching for internships or passively by checking suggested opportunities.

Companies are the organizations offering internship opportunities. They will be able to post internship vacancies, specifying the required skills, tasks, technologies, and terms. Companies can manage and edit their internship postings and review and evaluate student applications. The system will help companies identify students who best match their internship requirements through the Recommendation Engine. Companies will also be able to schedule interviews, manage the selection process, and check and respond to feedback provided by students.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- **Recommendation Engine:** System which manages the recommendation of students' CVs to companies and of internships to students

1.3.2 Acronyms

- S&C: Students&Companies

1.3.3 Abbreviations

- G*: goal
- WP*: world phenomena
- SP*: shared phenomena
- R*: functional requirement
- UC*: use case

1.4 Revision history

Revision	Date	Author(s)	Description
1.0	31.12.2024	L. Ilardi, M. Luraghi	Chapter 1, init Chapter 2
2.0	03.01.2024	L. Ilardi, M. Luraghi	continued Chapter 2, Chapter 4
3.0	06.01.2025	L. Ilardi, M. Luraghi	continued Chapter 2, init Chapter 5, Chapter 3
4.0	07.01.2025	L. Ilardi, M. Luraghi	finished Chapter 2, finished Chapter 5

1.5 Reference Documents

The document is based on the following materials:

- The RASD document
- The specifications RASD and DD assignment of the Software Engineering II course a.a. 2024/2025
- Course slides on Webeep

1.6 Document Structure

1. **Introduction:** it provides an overview of the project, focusing on the motivations and objectives behind its development. It also references the goals and requirements outlined in the RASD
2. **Architectural Design:** it presents the architecture of the system, describing it at different levels of granularity
3. **User Interface Design:** it showcases the wireframes that define the platform's user interface
4. **Requirements Traceability:** it explains how the objectives defined in the RASD are fulfilled by the interaction between the requirements and the architectural components detailed in this document
5. **Implementation, Integration, and Test Plan:** it describes the implementation, integration and testing details
6. **Effort Spent:** it shows the time spent to realize this document organized by section

Chapter 2

Architectural Design

2.1 Overview: High-level components and their interaction

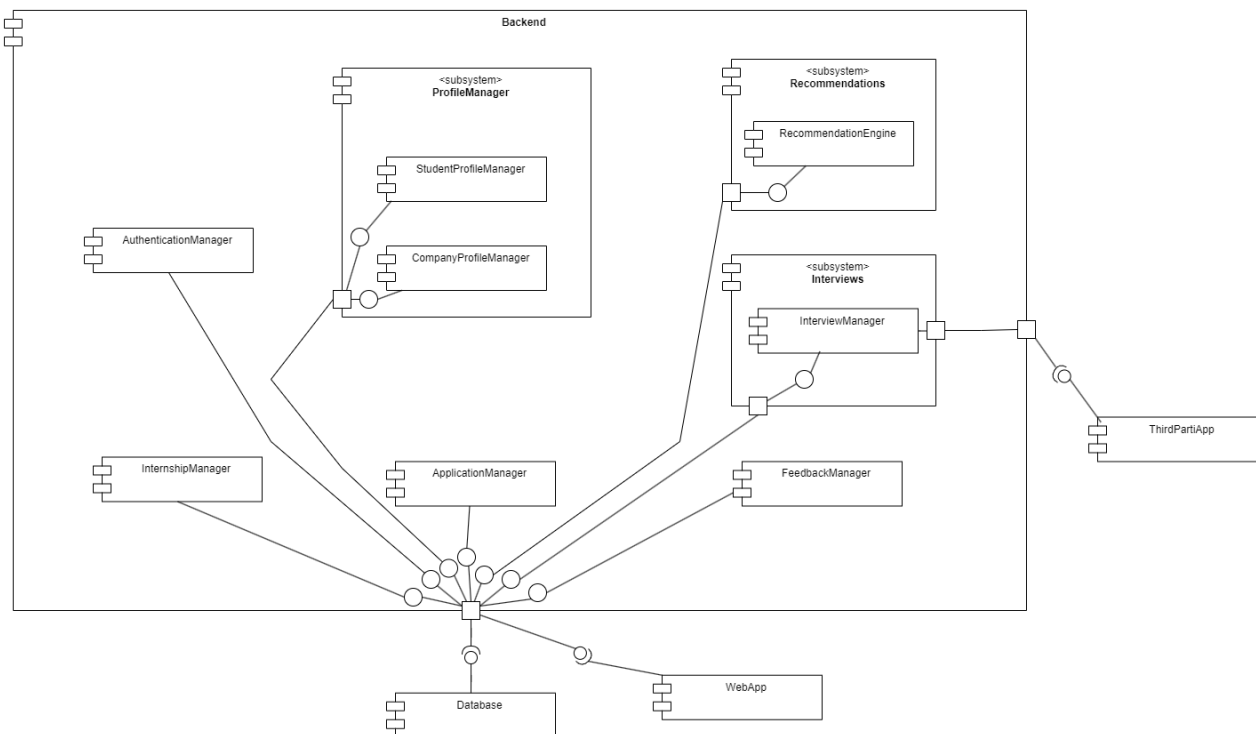
For the S&C platform, we will adopt a three-tier architecture, a widely used software application design that separates the system into three distinct layers:

1. **Presentation Layer:** this is the user interface, where users interact with the application
2. **Application Layer:** this layer handles data processing and application logic
3. **Data Layer:** this is responsible for storing and managing the application's data

The primary advantage of this architecture lies in its modularity. Each layer can be developed, maintained, and scaled independently by separate teams, ensuring greater flexibility and efficiency. Moreover, changes or scaling in one tier do not directly impact the others, making the system more adaptable to evolving requirements.

2.2 Component view

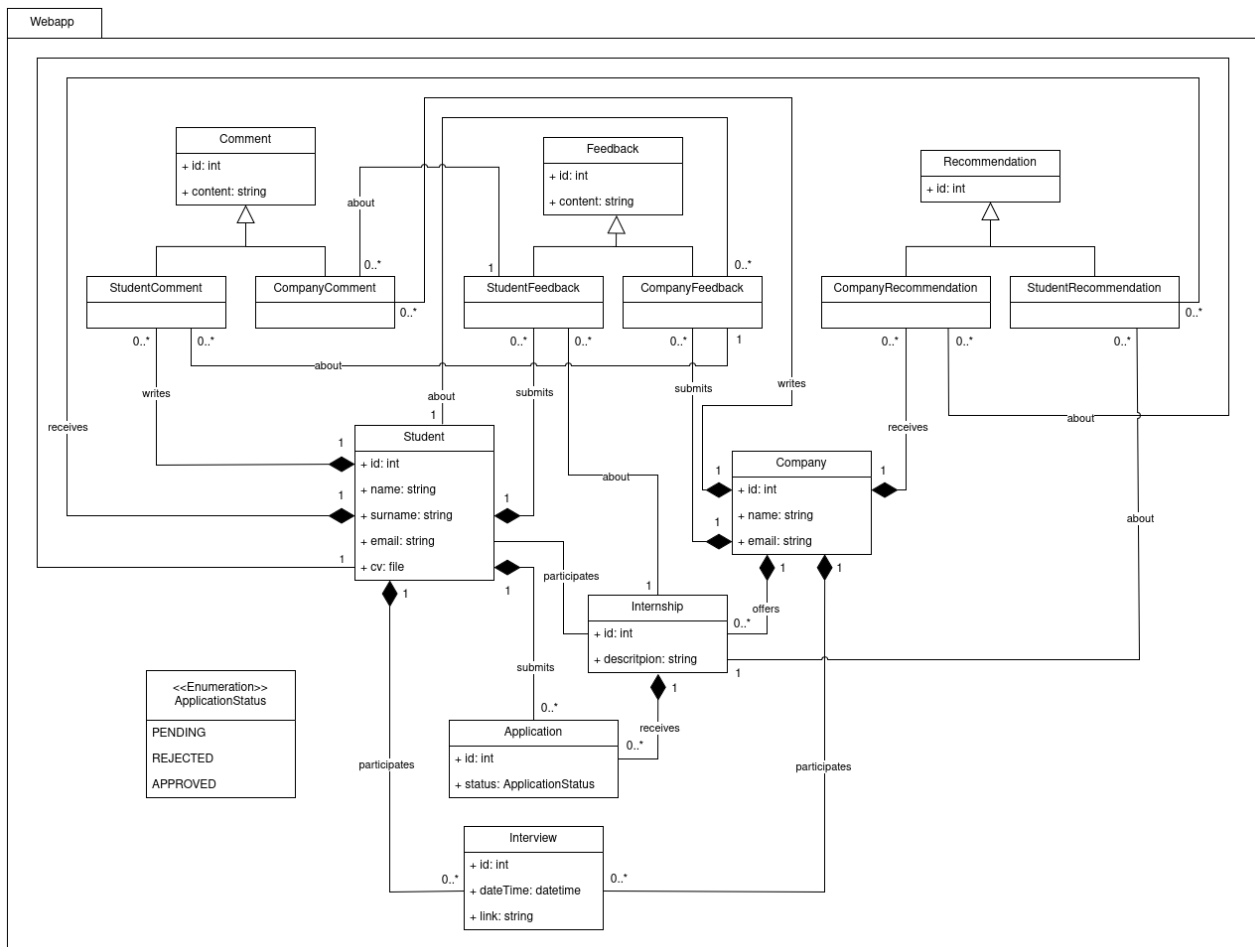
2.2.1 Component diagram



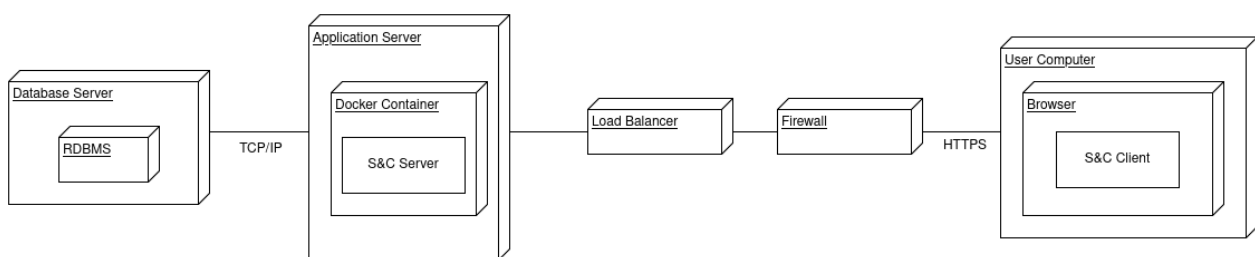
The components that make up the application are the following:

- **AuthenticationManager**: handles login and signup functionalities
- **InternshipManager**: manages internships, including creation, editing, and deletion
- **ApplicationManager**: handles internship applications, including submission, acceptance, and rejection
- **InterviewManager**: integrates with a third-party app for interviews and manages scheduling
- **StudentProfileManager**: manages student profiles, personal details, and CV uploads
- **CompanyProfileManager**: manages company profiles
- **FeedbackManager**: supports feedback submission and commenting
- **RecommendationEngine**: suggests internships and students based on preferences
- **WebApp**: serves as both the view and the controller. It routes user requests to the appropriate components, processes their responses, and sends the final view back to the end user
- **Database**: stores all of the application's data

2.2.2 Class diagram

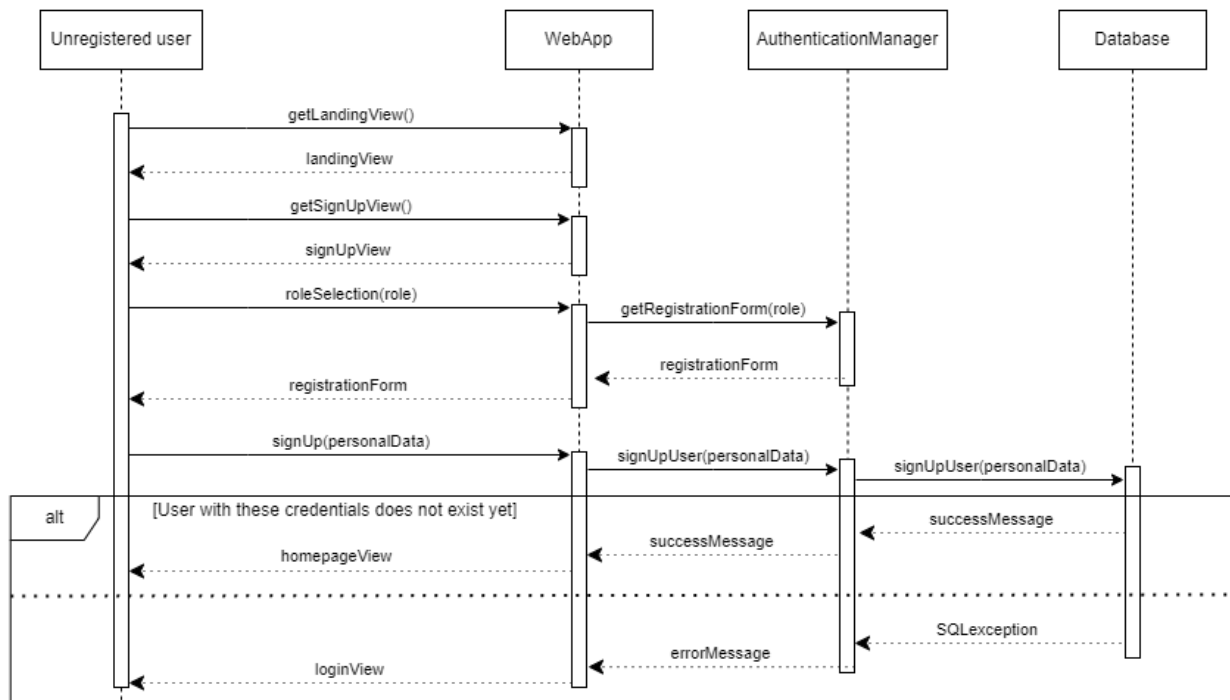


2.3 Deployment view

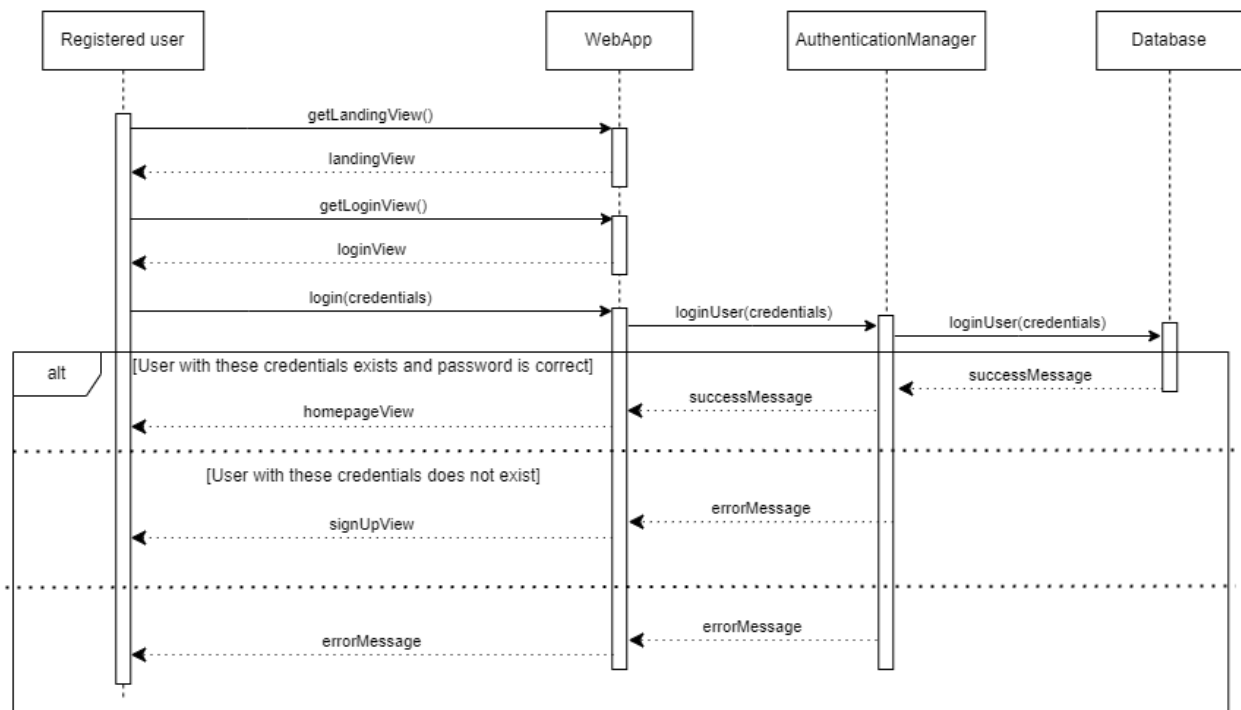


The client-side S&C app runs in the browser, and every client request is first processed by a firewall to detect and block any malicious activity. A load balancer prevents brute force attacks and ensures the server is not overwhelmed by excessive requests, protecting its CPU capacity. The application server, running in a Docker container, manages the server-side software architecture. The application server updates any changes to the database via TCP/IP. Running the server in a Docker container provides portability, scalability, and consistent environment management.

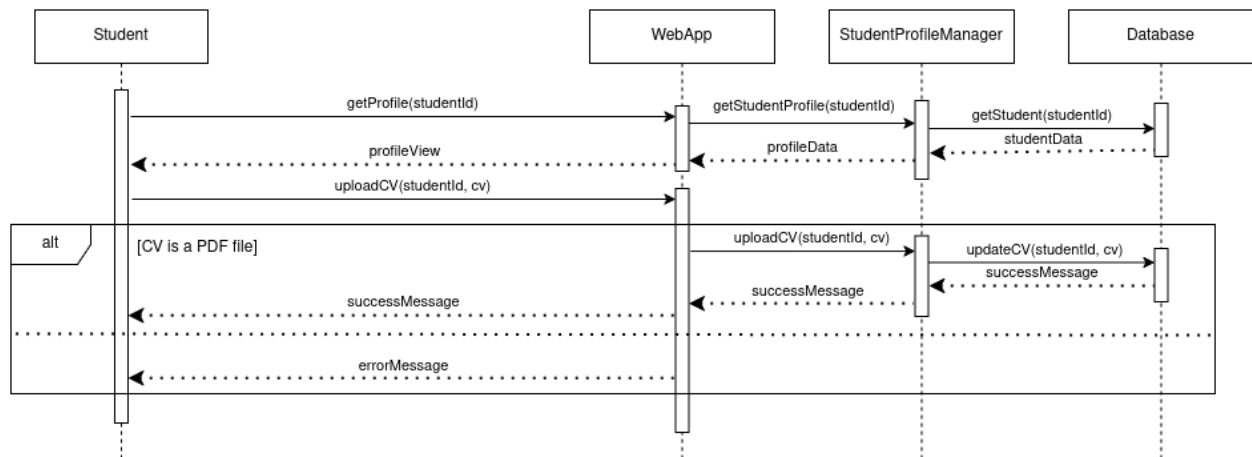
2.4 Runtime view



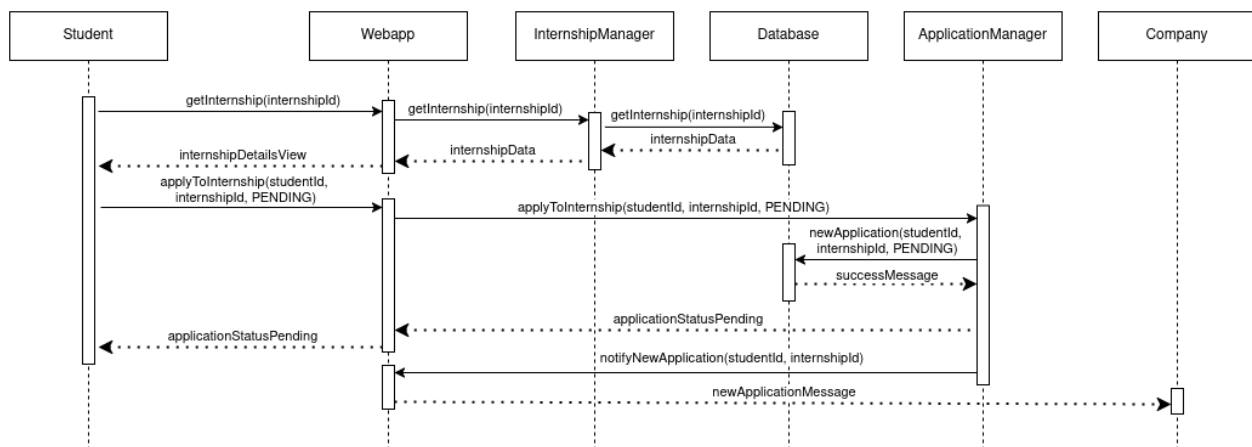
[UC1] User signs up



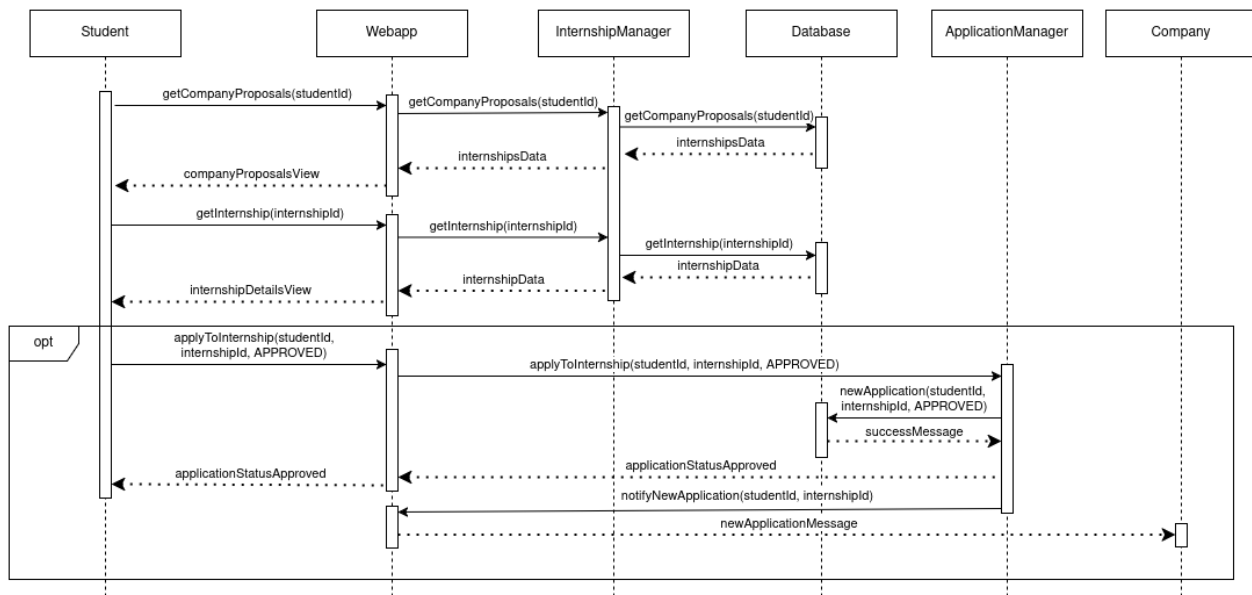
[UC2] User logs in



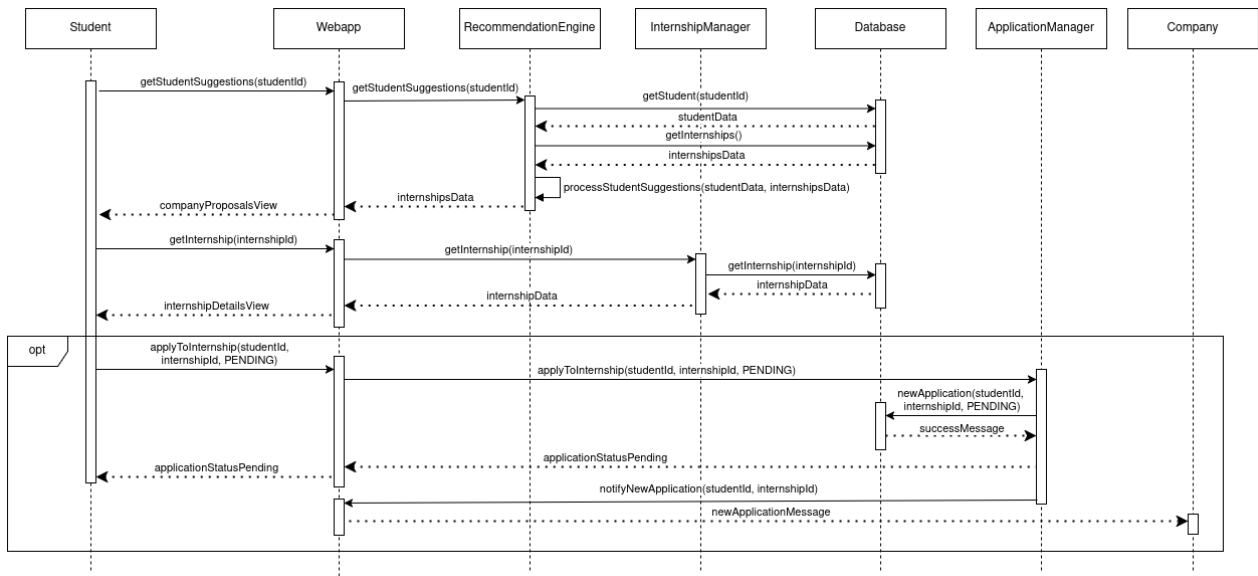
[UC3] Student uploads CV



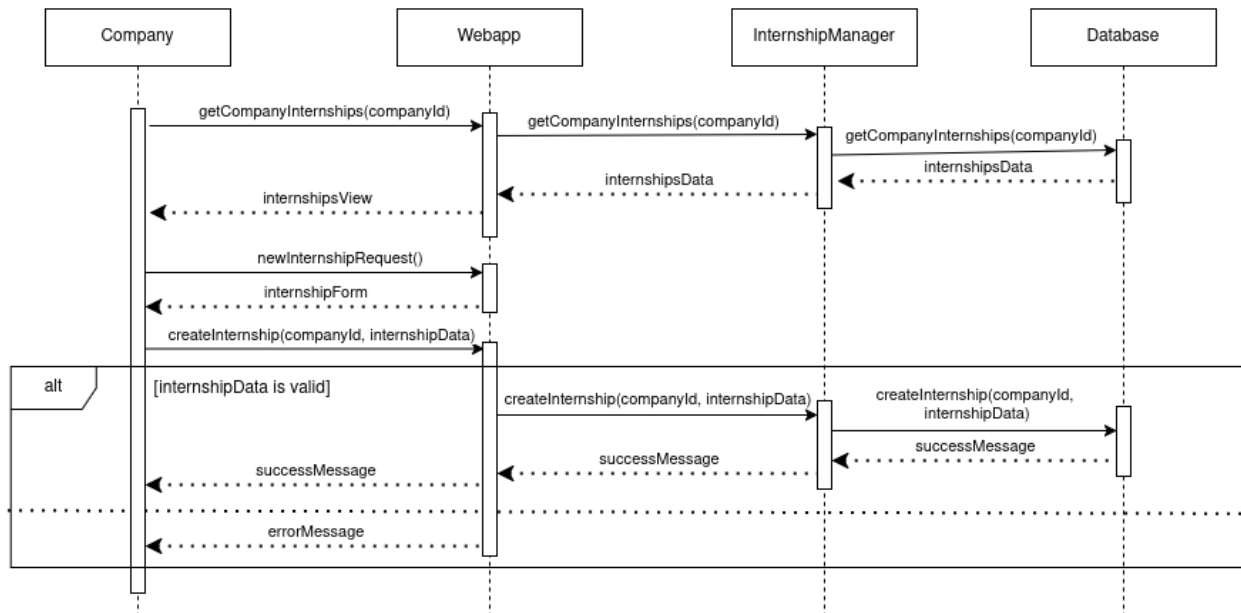
[UC4] Student applies to an internship he found



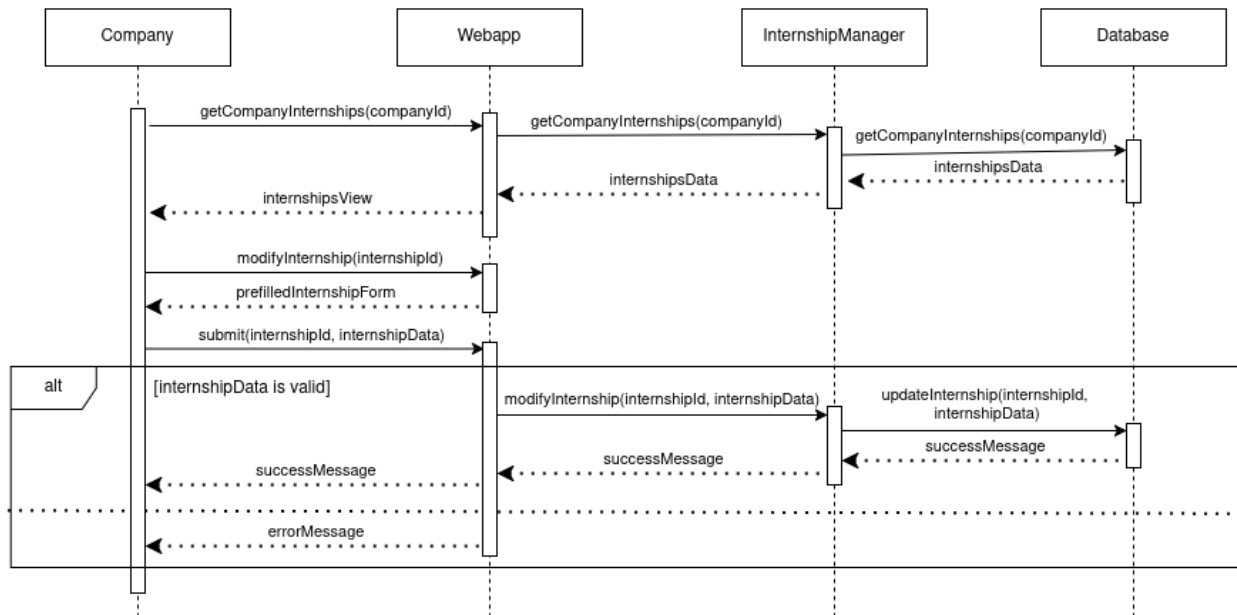
[UC5] Student checks an internship proposed by a company



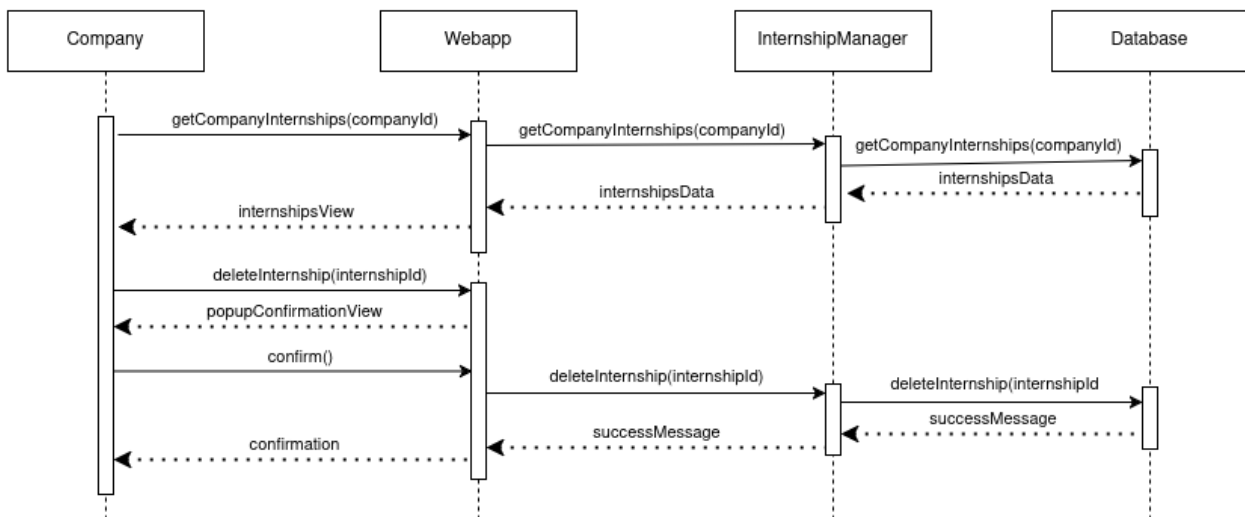
[UC6] Student checks an internship proposed by the system



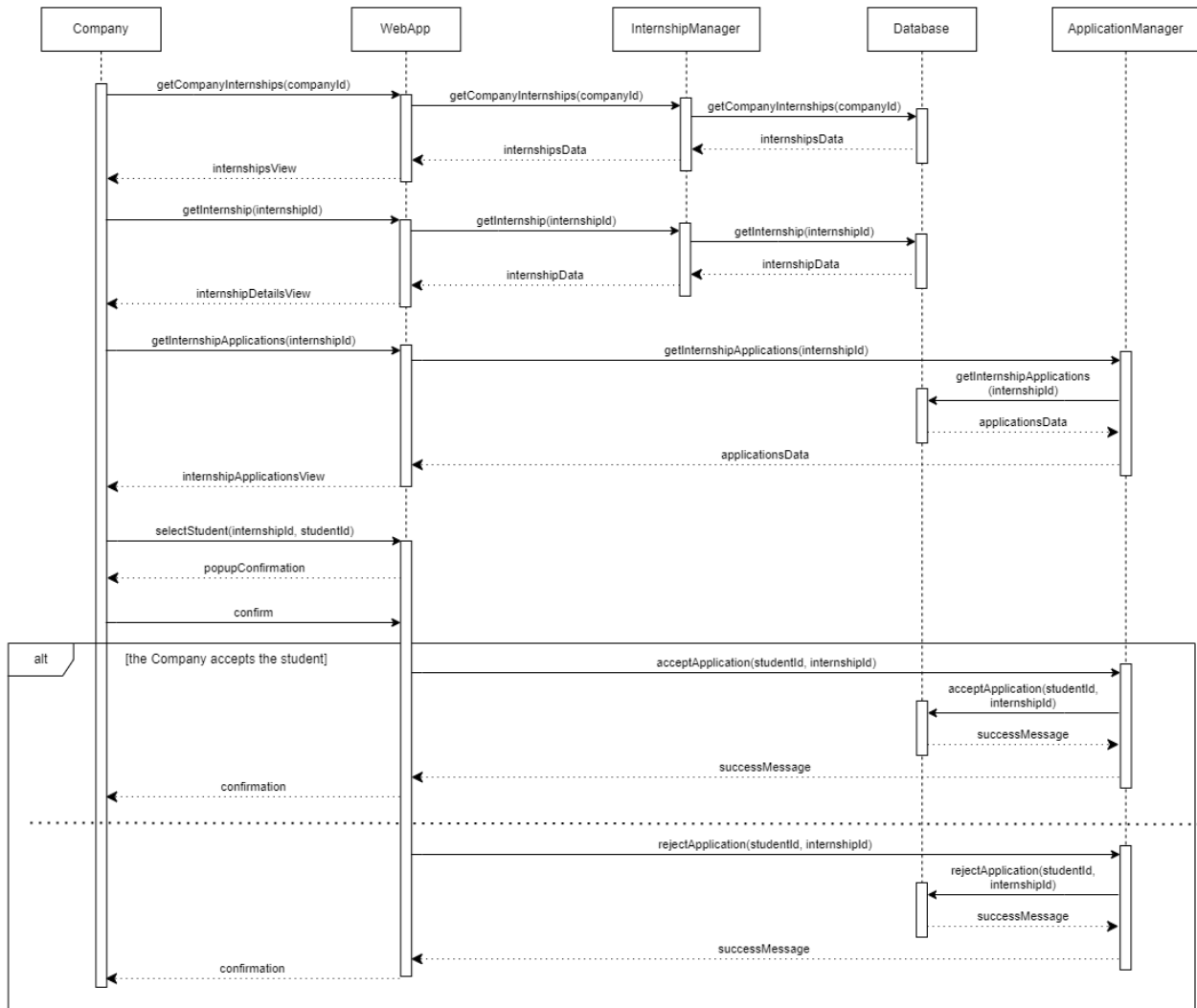
[UC7] Company creates an internship announcement



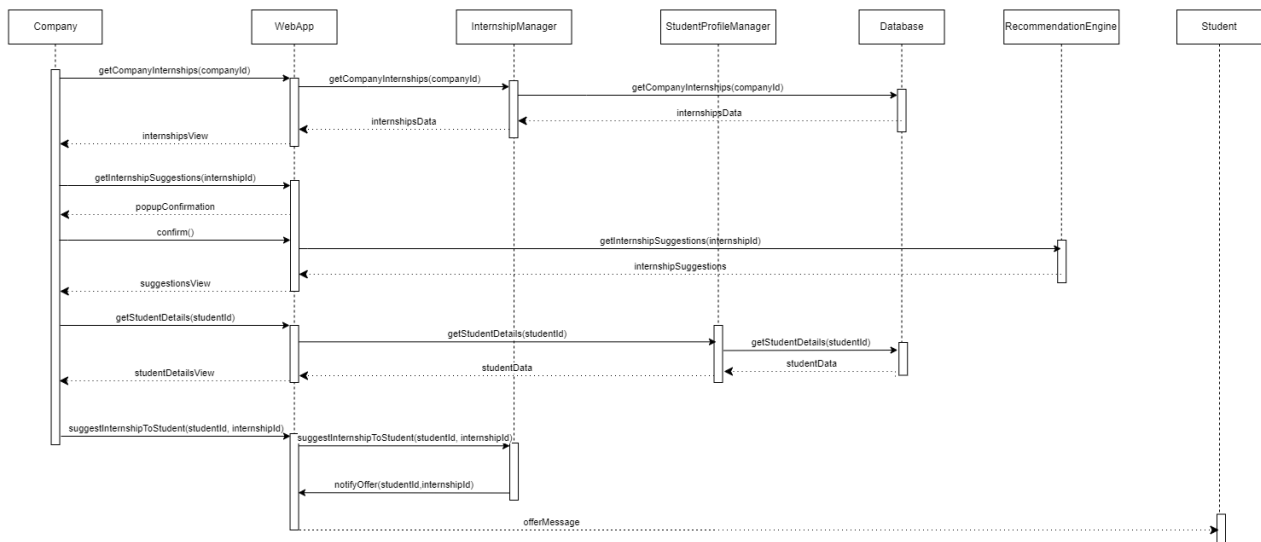
[UC8] Company modifies an internship announcement



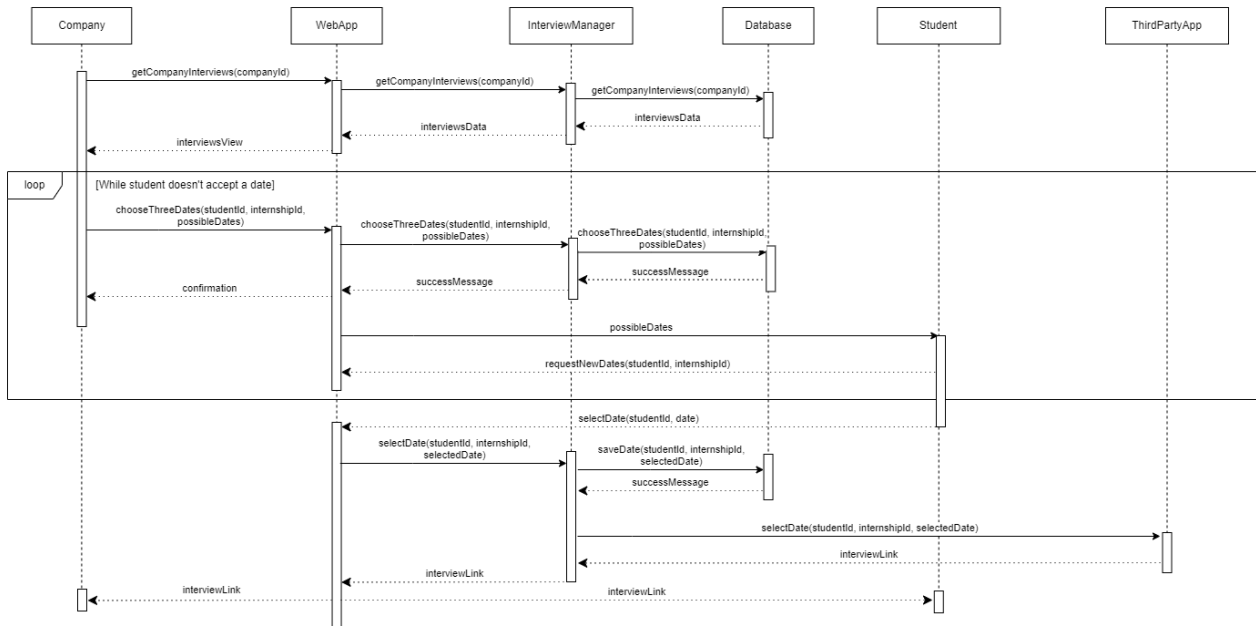
[UC9] Company deletes an internship announcement



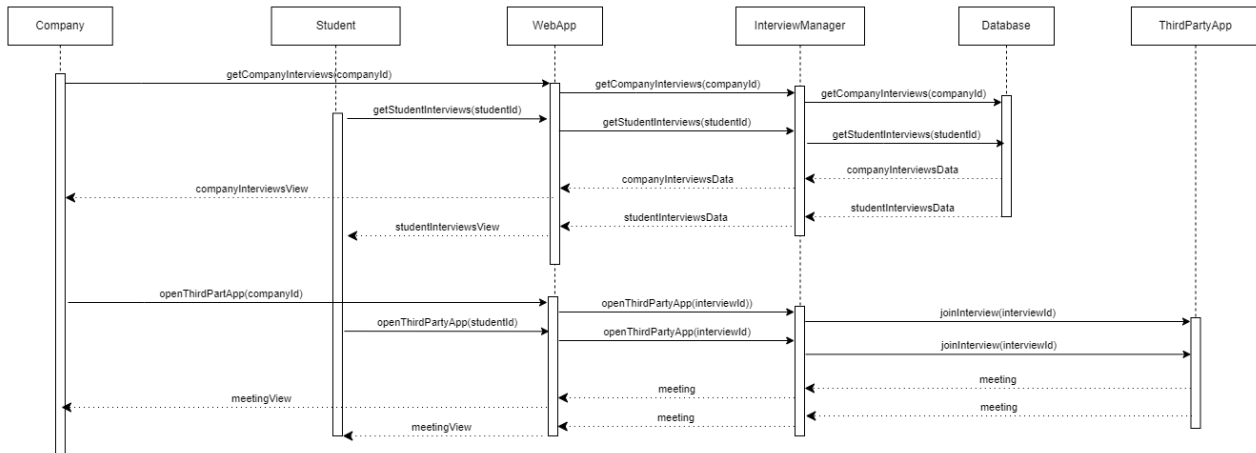
[UC10] Company reviews a student's application



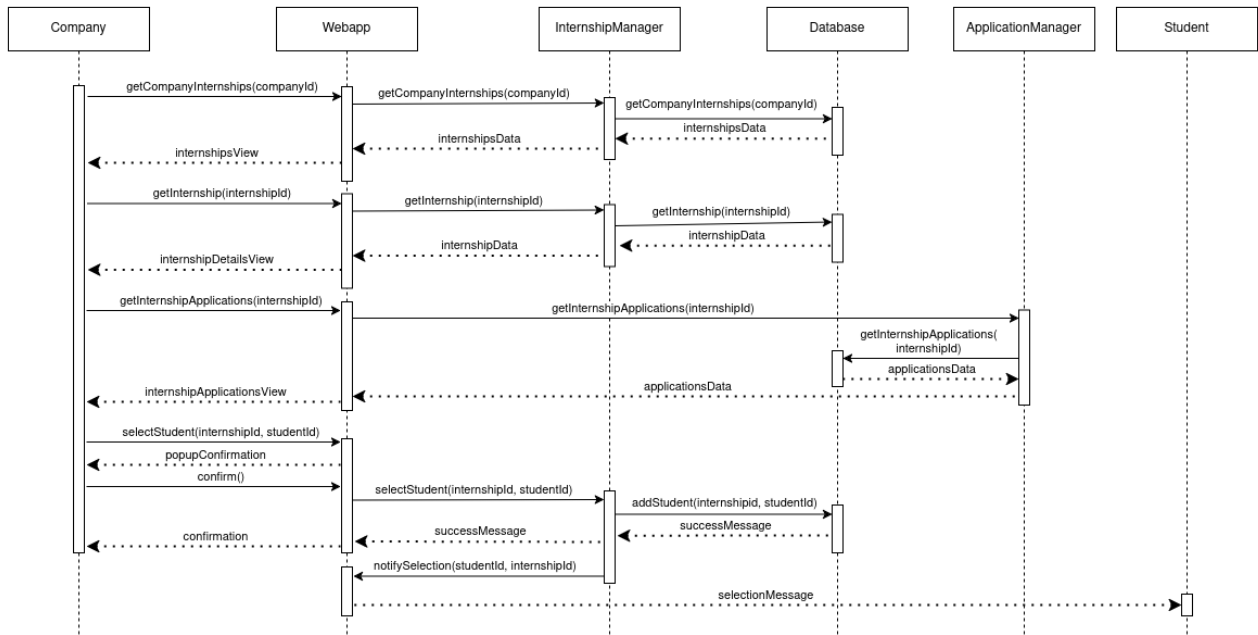
[UC11] Company offers an internship to a student



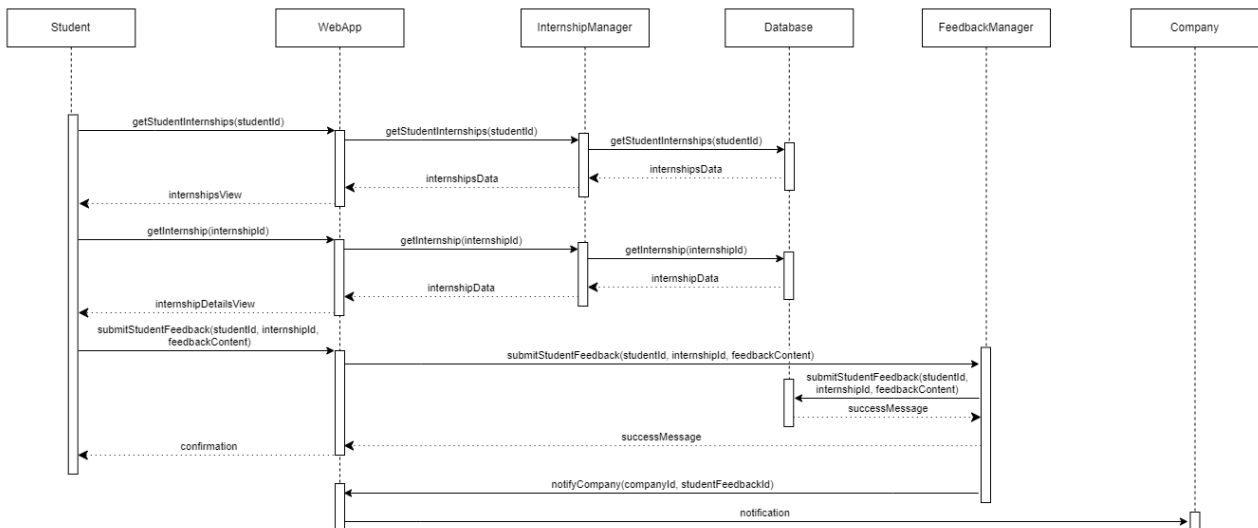
[UC12] Student and company set up the interview



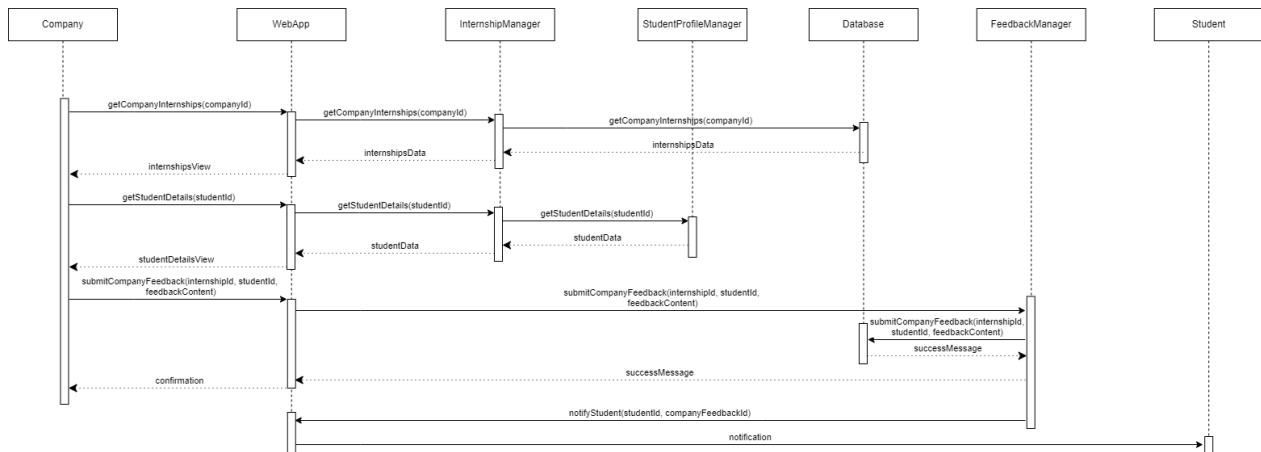
[UC13] Student and company meet for the interview



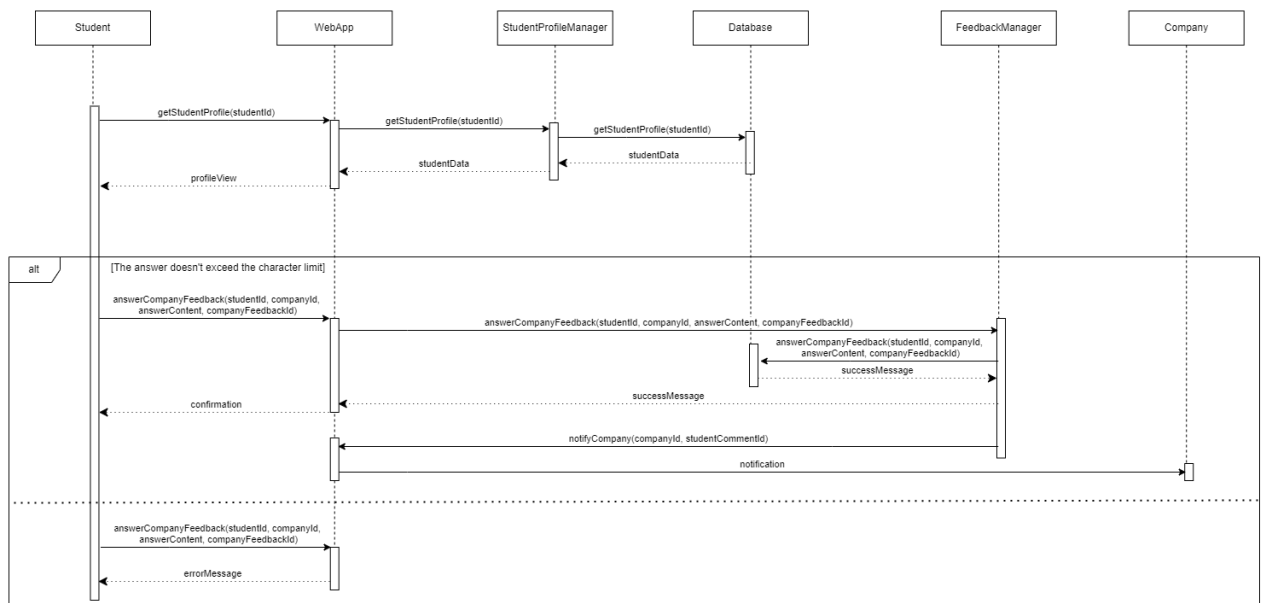
[UC14] Company admits a student to an internship



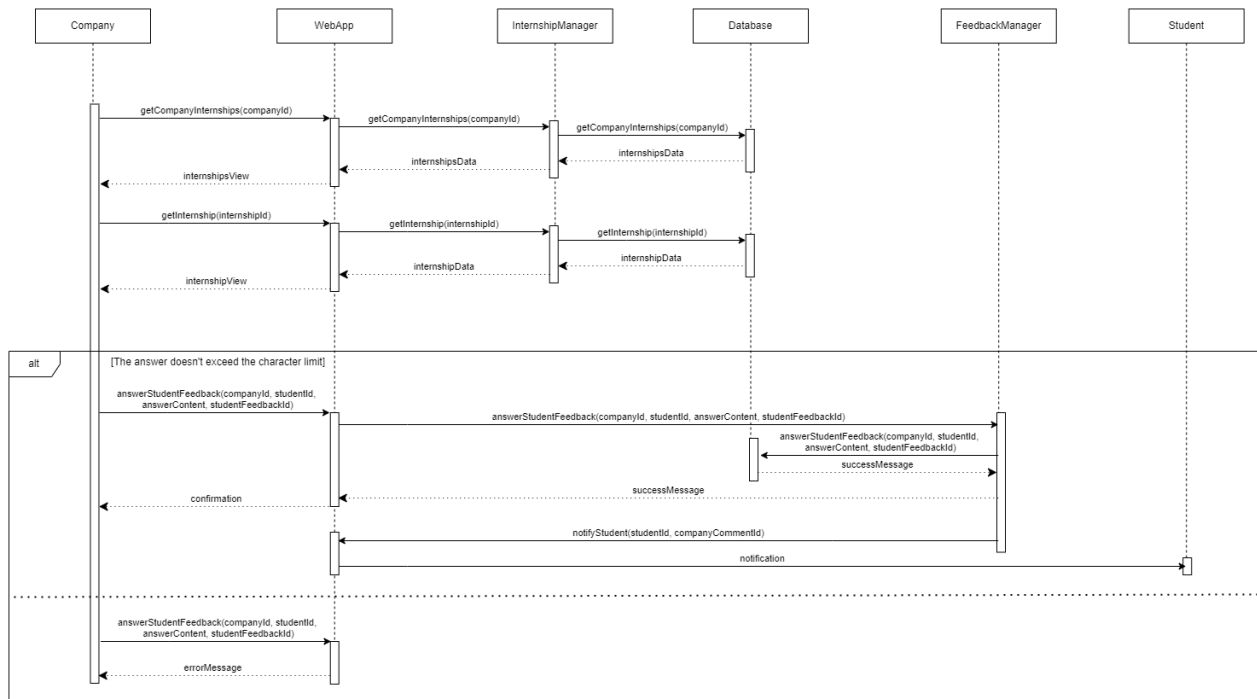
[UC15] Student provides feedback on an internship



[UC16] Company provides feedback on a student



[UC17] Student answers a comment about them



[UC18] Company answers a comment about an internship it offered

2.5 Component interfaces

AuthenticationManager

- getRegistrationForm(role: enum(COMPANY, STUDENT)) : json
- signUpUser(personalData: json)
- loginUser(credentials: json)

InternshipManager

- getCompanyInternships(companyId: int) : List[Internship]
- getStudentInternships(studentId: int) : List[Internship]
- getCompanyProposals(studentId: int) : List[Internship]
- getInternship(internshipId: int) : Internship
- getApplications(internshipId: int) : List[Application]
- selectStudent(internshipId: int, studentId: int)
- createInternship(companyId: int, internshipData: json)
- modifyInternship(internshipId: int, internshipData: json)
- deleteInternship(internshipId: int)
- suggestInternshipToStudent(studentId: int, internshipId: int)

ApplicationManager

- applyToInternship(studentId: int, internshipId: int, applicationStatus: ApplicationStatus)
- getStudentApplications(studentId: int) : List[Application]
- getInternshipApplications(internshipId: int) : List[Application]
- acceptApplication(studentId: int, internshipId: int)
- rejectApplication(studentId: int, internshipId: int)

InterviewManager

- getStudentInterviews(studentId: int) : List[Interview]
- getCompanyInterviews(companyId: int) : List[Interview]
- chooseThreeDates(studentId: int, internshipId: int, possibleDates: List[Date])
- selectDate(studentId: int, internshipId: int, selectedDate: Date)
- requestNewDates(studentId: int, internshipId: int)
- openThirdPartyApp(interviewId: int)
- joinInterview(interviewId: int)

StudentProfileManager

- updateStudentProfileInfo(studentId: int, data: json)
- uploadCv(studentId: int, cv: File)
- getStudentProfile(studentId: int) : json
- getStudentDetails(studentId: int) : json

CompanyProfileManager

- getCompanyProfile(companyId: int) : json
- updateCompanyProfileInfo(companyId: int, data : json)

FeedbackManager

- submitStudentFeedback(studentId: int, internshipId: int, feedbackContent: String)
- submitCompanyFeedback(internshipId: int, studentId: int, feedbackContent: String)
- answerStudentFeedback(companyId: int, studentId: int, answerContent: String, studentFeedbackId: int)
- answerCompanyFeedback(studentId: int, companyId: int, answerContent: String, companyFeedbackId: int)

RecommendationEngine

- getStudentSuggestions(studentId: int) : List[StudentRecommendation]
- getInternshipSuggestions(internshipId: int) : List[CompanyRecommendation]

Webapp

- notifyOffer(studentId: int, internshipId: int)
- notifySelection(studentId: int, internshipId: int)
- notifyUpdatedApplicationStatus(studentId: int, internshipId: int, applicationStatus: ApplicationStatus)
- notifyNewApplication(studentId: int, internshipId: int)
- notifyStudent(studentId: int, companyFeedbackId: int)
- notifyCompany(companyId: int, studentFeedbackId: int)
- notifyStudent(studentId: int, companyCommentId: int)
- notifyCompany(companyId: int, studentCommentId: int)

2.6 Selected architectural styles and patterns

2.6.1 3-tier architecture

The system follows a 3-tier architectural style due to its numerous benefits, with modularization into three independent layers being the most significant. These layers are:

- **Client (Presentation Tier):** handles the user interface, presented as a website
- **Application Server (Middle Tier):** is responsible for implementing the business logic and runs the S&C server in a Docker container
- **Database (Backend Tier):** manages data storage and retrieval, operating on a database management system (DBMS)

2.6.2 Model View Controller pattern

The Model-View-Controller pattern is a robust choice for implementing the S&C platform. It is structured as follows:

- **Model:** manages the application's data, business logic, and rules
- **View:** handles the presentation of data to the end user
- **Controller:** acts as an intermediary between the Model and the View, processing user input and ensuring proper data flow and format between components

For the S&C platform, all components except the Webapp will serve as the Model. The Webapp will handle both the View and Controller aspects, facilitating communication with the Model components and rendering the user interface by translating the data accordingly.

This division ensures a clear separation of concerns, enhancing scalability, promoting reusability, and maintaining each module's independence, simplifying maintenance and future development.

2.7 Other design decisions

2.7.1 Availability

Load balancing is introduced to create a highly available system capable of handling many requests simultaneously. Replication is also crucial to prevent a single point of failure.

2.7.2 Security

Security is managed through a firewall, it acts as a barrier between the internal system and external threats, ensuring that only authorized traffic is allowed while safeguarding the system from malicious activities.

Chapter 3

User Interface Design

Student's View

Company ProposalsSuggestionsApplicationsInternshipsInterviewsJohn Doe

Internship Announcements

Frontend Developer
Company: TechCorp

Data Analyst
Company: DataMinds

Software Engineer
Company: InnovateX

Marketing Intern
Company: MarketHub

Backend Developer
Company: CodeBase

UX/UI Designer
Company: CreativeWorks

Cybersecurity Intern
Company: SecureTech

Cloud Engineer
Company: SkySolutions

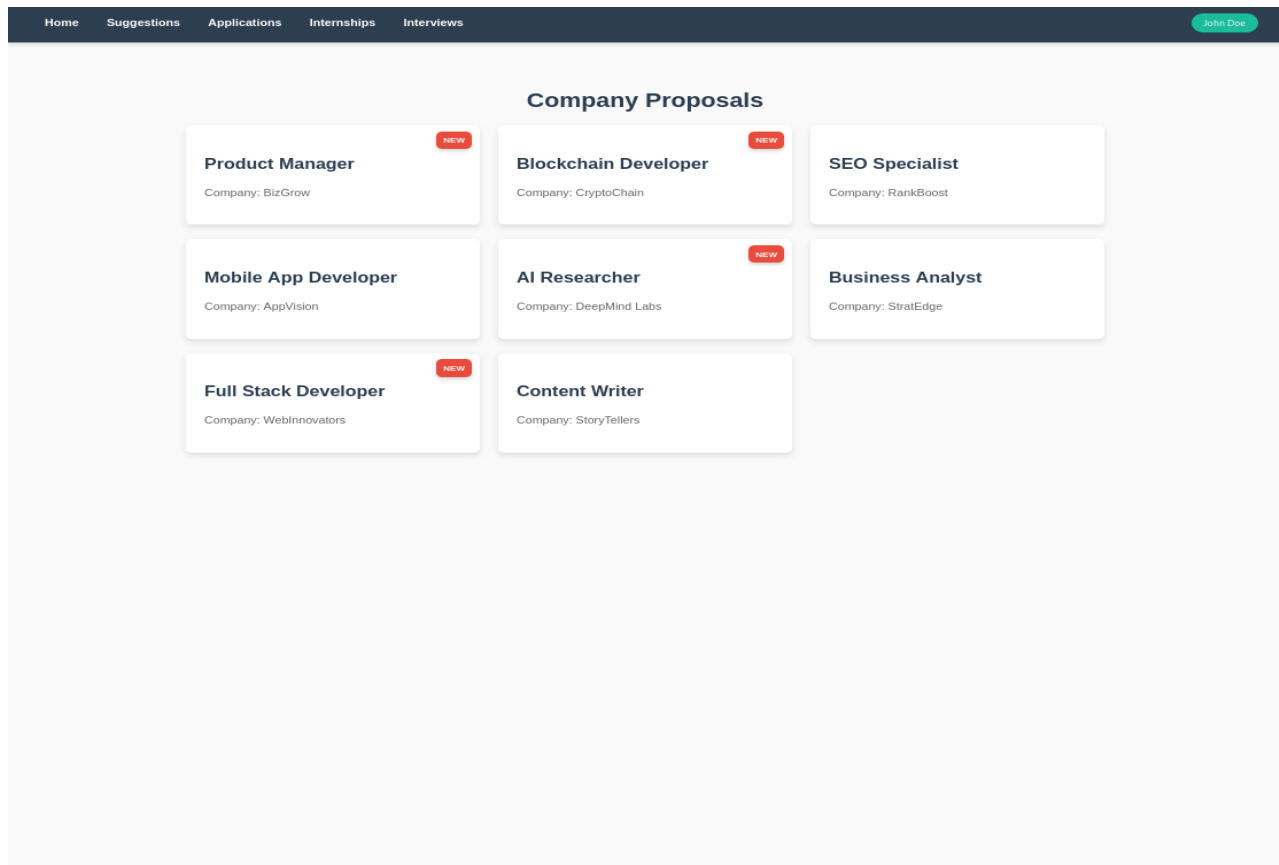
Machine Learning Intern
Company: AIPros

Game Developer
Company: PixelCraft

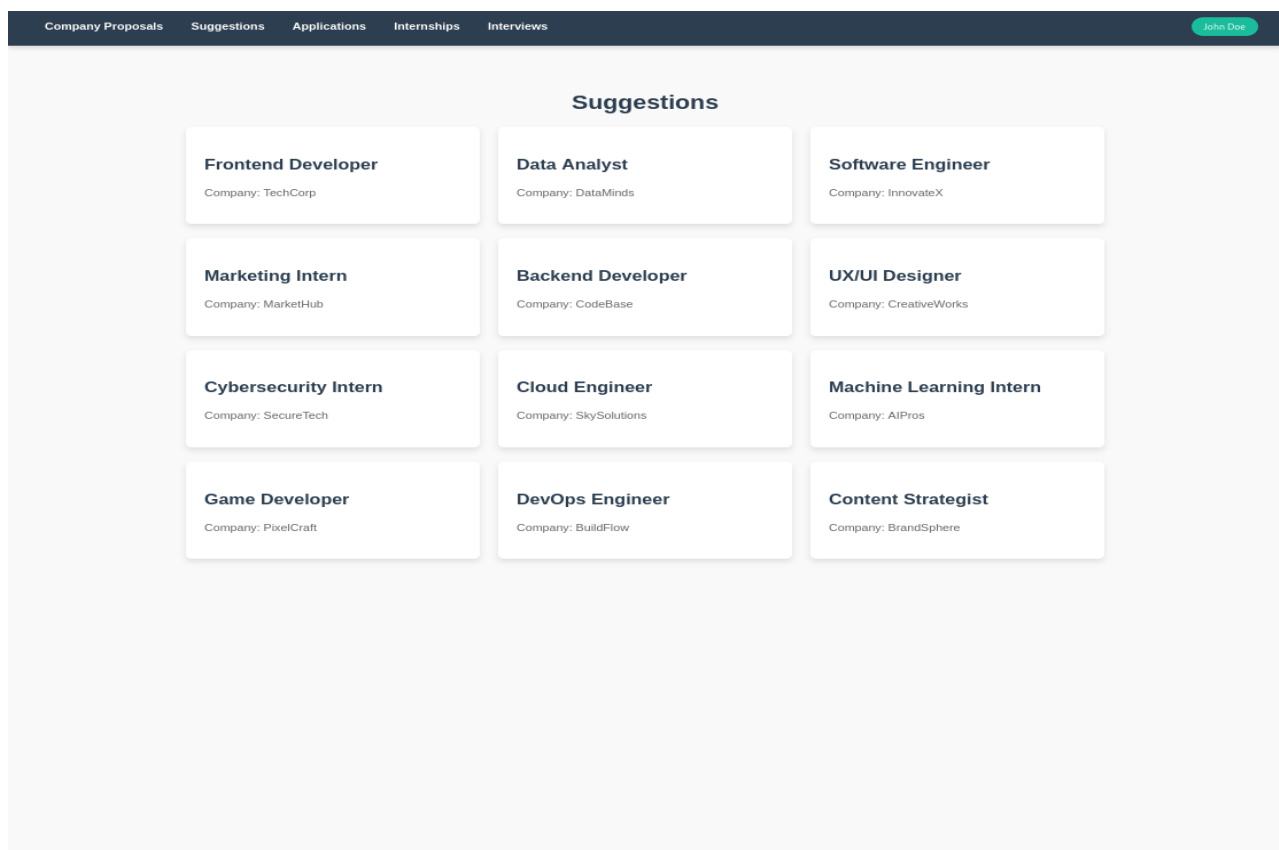
DevOps Engineer
Company: BuildFlow

Content Strategist
Company: BrandSphere

Homepage



Company Proposals



Suggestions

HomeCompany ProposalsSuggestionsInternshipsInterviews

John Doe

My Applications

Proposed Dates for Incoming Interviews

Frontend Developer

Company: TechCorp

01-202502-202503-2025

Reschedule

Data Analyst

Company: DataMinds

06-202507-202508-2025

Reschedule

Software Engineer

Company: InnovateX

11-202512-202501-2026

Reschedule

Frontend Developer at TechCorp

Period: 06-2023 / 08-2024

Status: APPROVED

UX/UI Designer at CreativeWorks

Period: 05-2024 / 10-2024

Status: PENDING

Cloud Engineer at SkySolutions

Period: 04-2024 / 09-2024

Status: REJECTED

Software Engineer at InnovateX

Applications

HomeCompany ProposalsSuggestionsApplicationsInterviews

John Doe

Internships

Ongoing Internships

Frontend Developer

Company: TechCorp

Period: 06-2023 / 08-2024

Past Internships

Data Analyst

Company: DataMinds

Period: 01-2022 / 12-2022

Software Engineer

Company: InnovateX

Period: 03-2021 / 11-2021

UX/UI Designer

Company: CreativeWorks

Period: 05-2020 / 10-2020

Marketing Intern

Company: MarketHub

Period: 02-2019 / 08-2019

Cloud Engineer

Company: SkySolutions

Period: 04-2018 / 12-2018

Cybersecurity Intern

Company: SecureTech

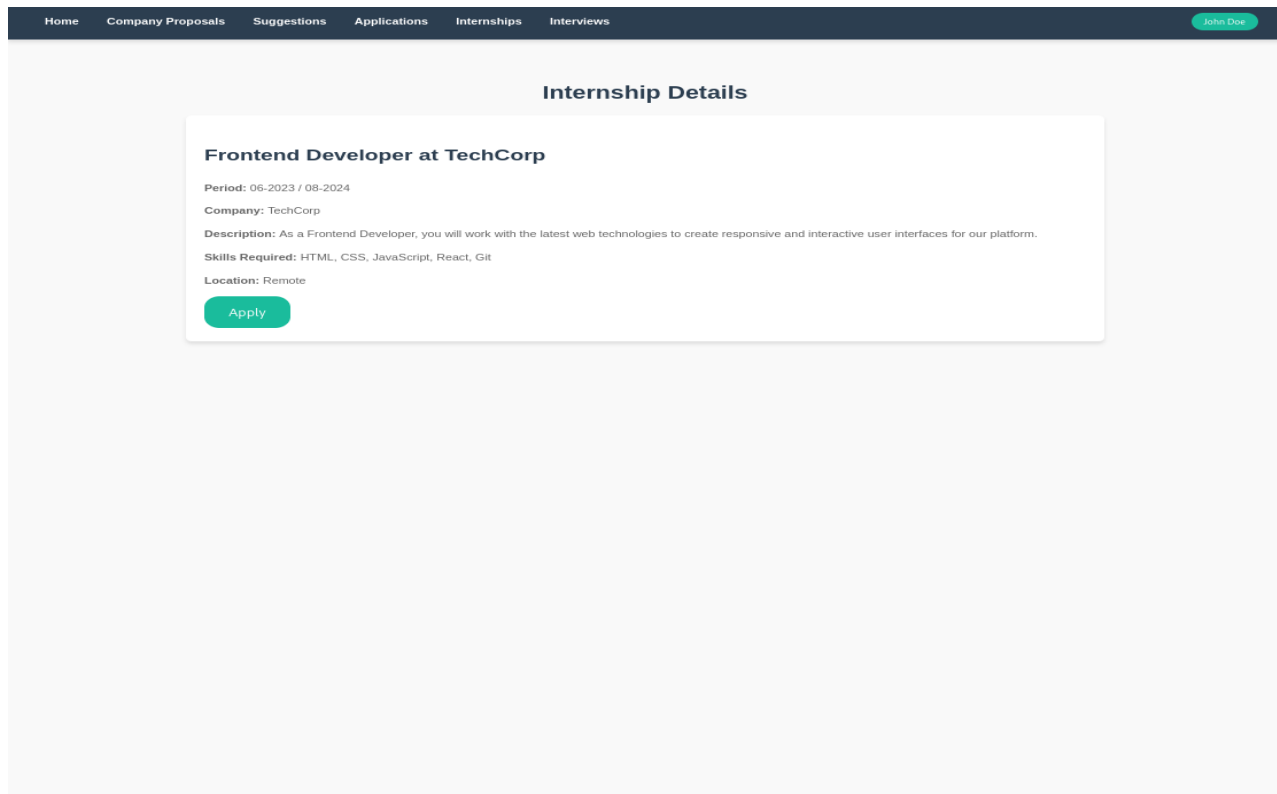
Period: 01-2017 / 09-2017

Business Analyst

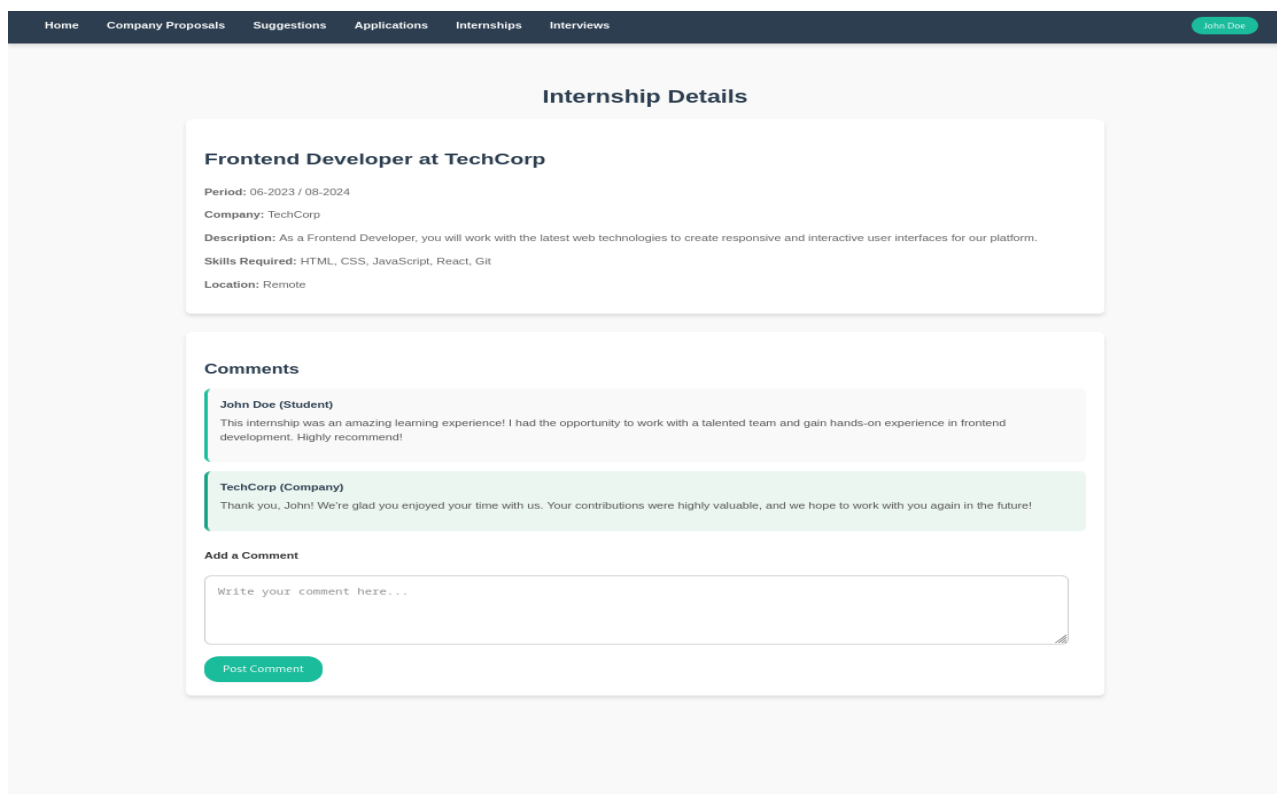
Company: StratEdge

Period: 07-2016 / 12-2016

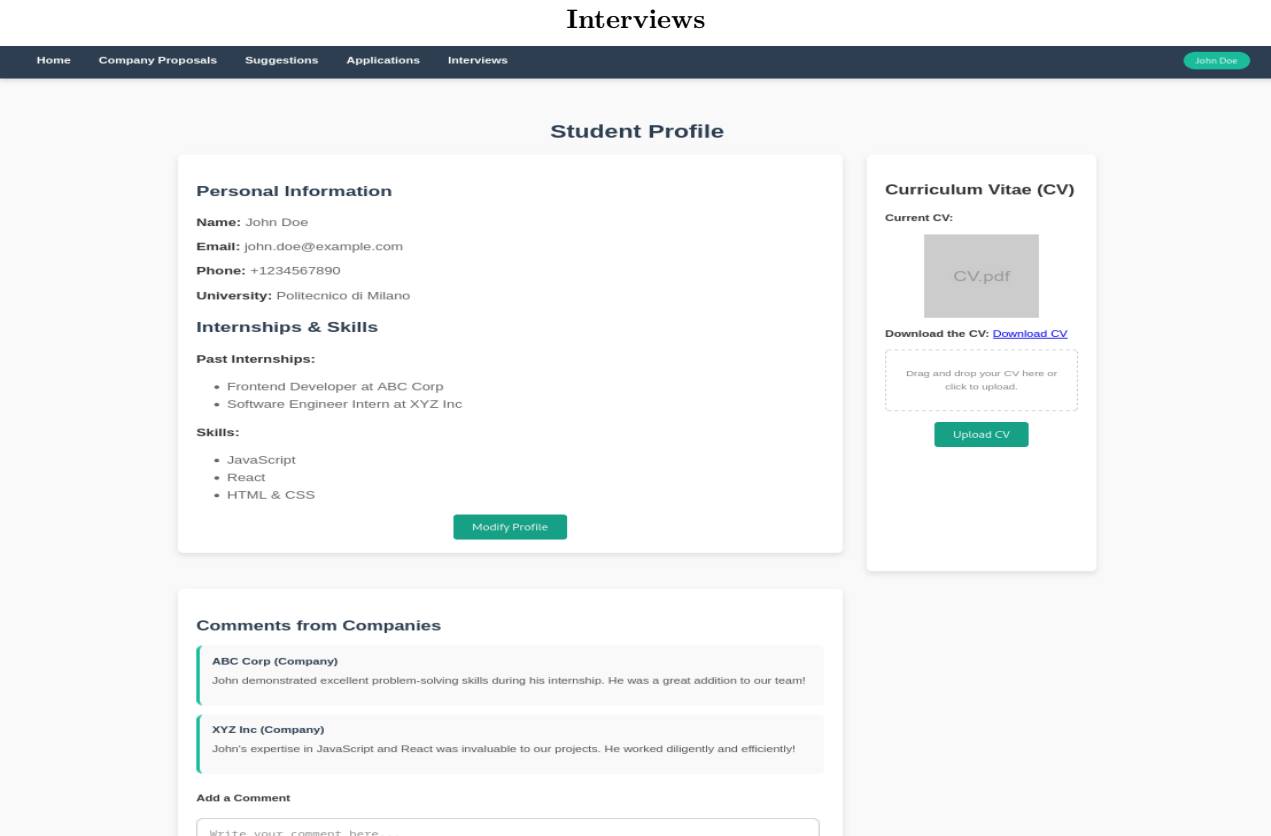
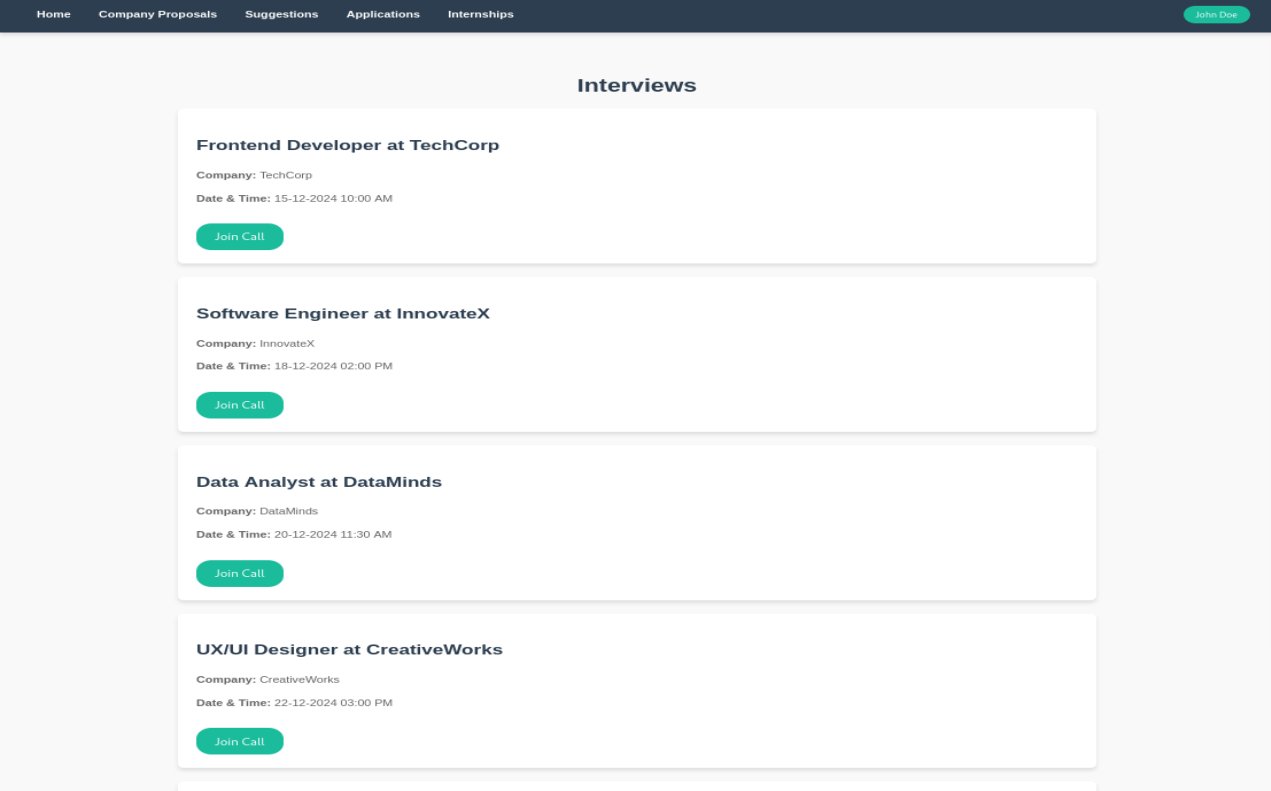
Internships



Available internship details



Past internship details



Profile

Company's View

[Home](#) [Internships](#) [Interviews](#) [TechCorp](#)

Students

Jane Smith
Email: jane.smith@email.com
Phone: +1234567890
University: Tech University
Skills: JavaScript, React, Node.js
[View CV](#)

John Doe
Email: john.doe@email.com
Phone: +0987654321
University: Code Academy
Skills: Python, Data Analysis, SQL
[View CV](#)

Alice Green
Email: alice.green@email.com
Phone: +1122334455
University: Innovation College
Skills: Java, Spring Boot, Microservices
[View CV](#)

Bob Brown
Email: bob.brown@email.com
Phone: +2233445566
University: Code Institute
Skills: HTML, CSS, Web Design
[View CV](#)

Charlie White
Email: charlie.white@email.com
Phone: +3344556677
University: Global Tech University
Skills: C++, Machine Learning
[View CV](#)

Homepage

[Home](#) [Interviews](#) [TechCorp](#)

Internships

New

Frontend Developer
Period: 06-2023 / 08-2024

Data Analyst
Period: 01-2022 / 12-2022

Software Engineer
Period: 03-2021 / 11-2021

UX/UI Designer
Period: 05-2020 / 10-2020

Marketing Intern
Period: 02-2019 / 08-2019

Cloud Engineer
Period: 04-2018 / 12-2018

Cybersecurity Intern
Period: 01-2017 / 09-2017

Business Analyst
Period: 07-2016 / 12-2016

Internships

[Home](#) [Internships](#) [Interviews](#) [TechCorp](#)

Create New Internship

Internship Title:

Internship Title

Internship Description:

Describe the internship...

Required Skills:

Skills (comma separated)

Submit

New internship form

[Home](#) [Internships](#) [Interviews](#) [TechCorp](#)

Internship Details

Frontend Developer

Period: 06-2023 / 08-2024

Description: As a Frontend Developer, you will work with the latest web technologies to create responsive and interactive user interfaces for our platform.

Skills Required: HTML, CSS, JavaScript, React, Git

Location: Remote

[Applications](#) [Suggestions](#) [Modify](#) [Delete](#)

Comments

John Doe (Student)

This internship was an amazing learning experience! I had the opportunity to work with a talented team and gain hands-on experience in frontend development. Highly recommend!

TechCorp (Company)

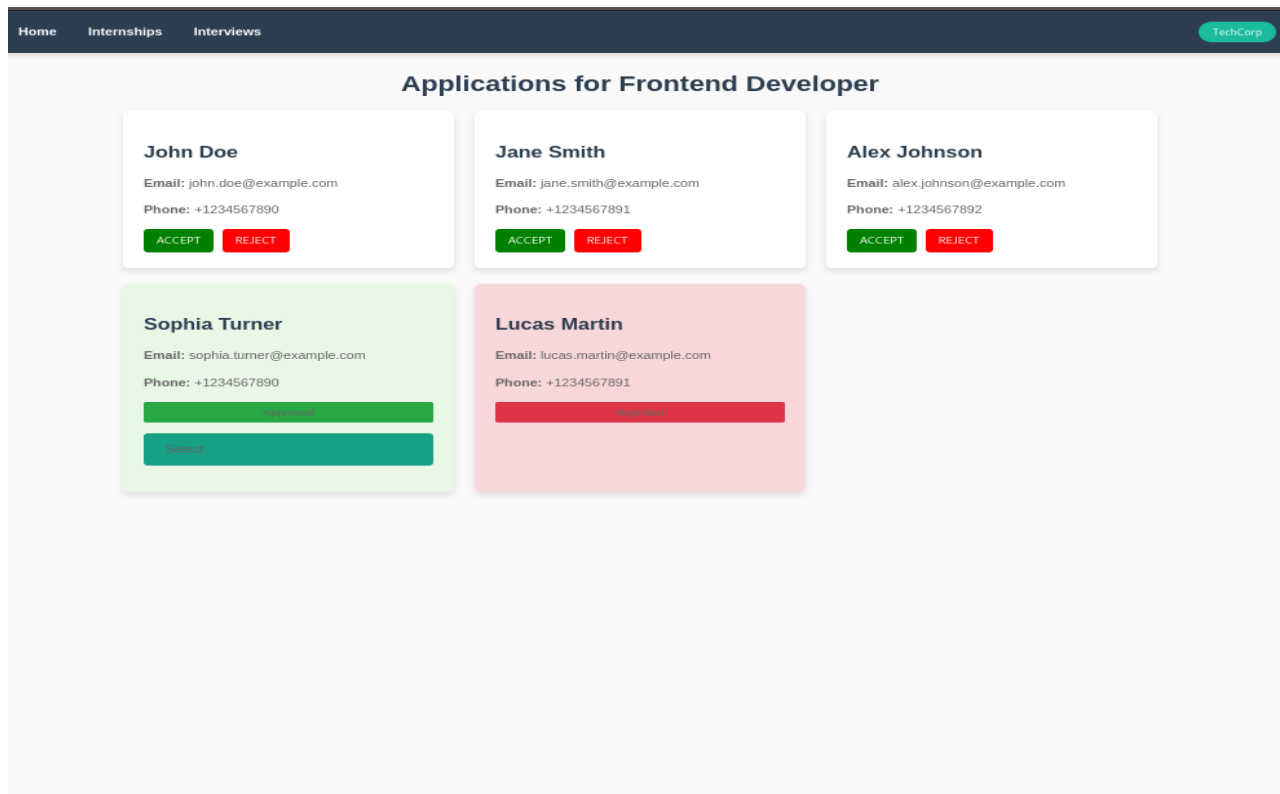
Thank you, John! We're glad you enjoyed your time with us. Your contributions were highly valuable, and we hope to work with you again in the future!

Add a Comment

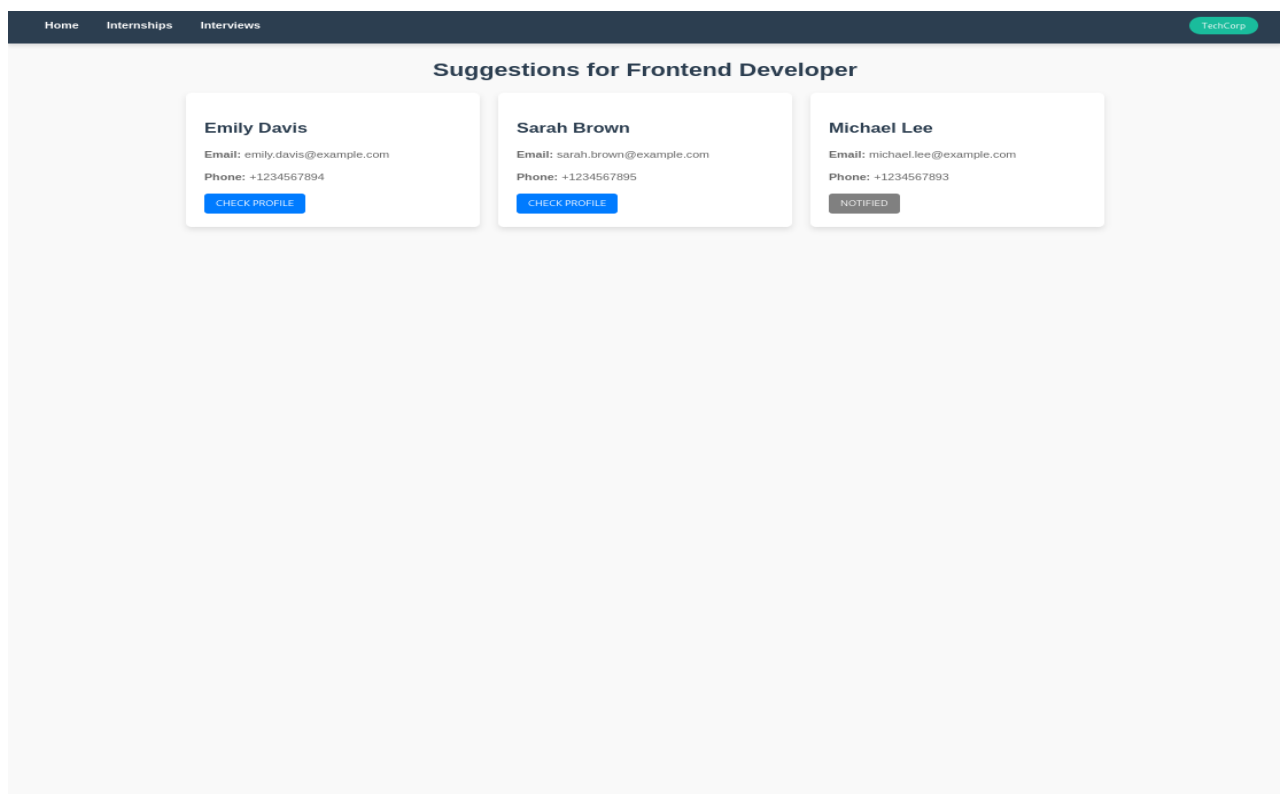
Write your comment here...

Post Comment

Internship details



Internship applications



Internship suggestions

HomeInternshipsTechCorp

Proposed Dates and Times for Incoming Interviews

Frontend Developer

Student Name: John Doe

Choose a date: mm / dd / yyyy

Choose a time: -- : -- --

Choose a date: mm / dd / yyyy

Choose a time: -- : -- --

Choose a date: mm / dd / yyyy

Choose a time: -- : -- --

Submit

Software Engineer

Student Name: Jane Smith

Choose a date: mm / dd / yyyy

Choose a time: -- : -- --

Choose a date: mm / dd / yyyy

Choose a time: -- : -- --

Choose a date: mm / dd / yyyy

Choose a time: -- : -- --

Submit

Data Analyst

Date & Time: 20-12-2024 11:30 AM

Student Name: Alice Green

Join Call

UX/UI Designer

Date & Time: 22-12-2024 03:00 PM

Student Name: Emily Davis

Join Call

Cloud Engineer

Date & Time: 25-12-2024 09:00 AM

Student Name: Charlie Mill

Interviews

HomeInternshipsInterviewsTechCorp

Student Profile

Personal Information

Name: John Doe

Email: john.doe@example.com

Phone: +1234567890

University: Politecnico di Milano

Internships & Skills

Past Internships:

Frontend Developer at ABC Corp

Software Engineer Intern at XYZ Inc

Skills:

JavaScript

React

HTML & CSS

Curriculum Vitae (CV)

Current CV:

CV.pdf

Download the CV: [Download CV](#)

NOTIFY

Comments from Companies

ABC Corp (Company)

John demonstrated excellent problem-solving skills during his internship. He was a great addition to our team!

XYZ Inc (Company)

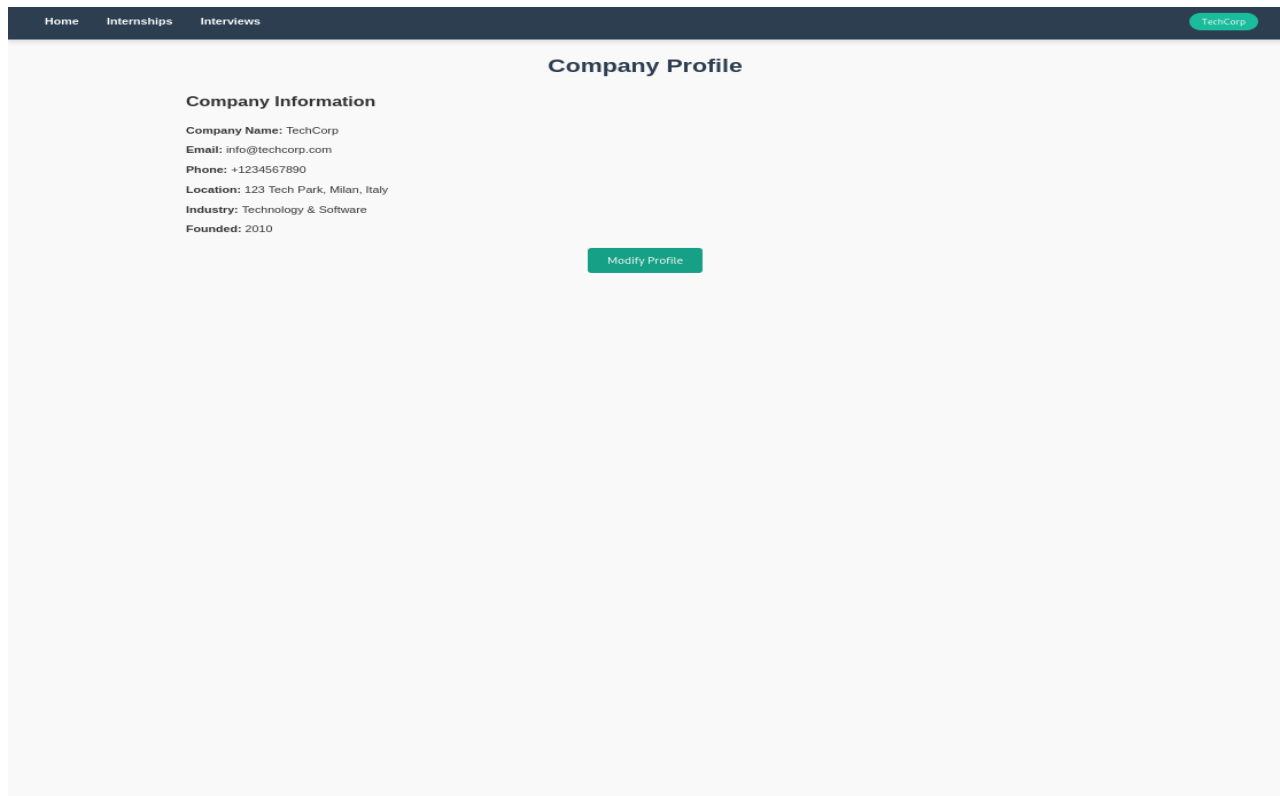
John's expertise in JavaScript and React was invaluable to our projects. He worked diligently and efficiently!

Add a Comment

Write your comment here...

Post Comment

Student profile



Profile

Chapter 4

Requirements Traceability

Requirements	Sign up and log in [R1] The System shall allow new users to sign up by providing their personal information [R2] The System shall allow registered users to log in by providing a username and a password
Components	Webapp AuthenticationManager Database

Requirements	Student CV management [R3] The System shall allow Students to upload a CV
Components	Webapp StudentProfileManager Database

Requirements	Internship announcement posting [R4] The System shall allow Companies to create internship announcement posts containing all the needed details [R5] The System shall allow Companies to modify existing internship announcement posts updating the details [R6] The System shall allow Companies to delete existing internship announcement posts
Components	Webapp InternshipManager Database

Requirements	Internship application [R7] The System shall allow Students to view the details of any internship [R8] The System shall allow Students to browse the suggested internships [R9] The System shall allow Students to apply for an internship [R10] The System shall allow Students to accept an internship proposal from a Company [R11] The System shall allow Students to check the status of the applications they've submitted
Components	Webapp InternshipManager ApplicationManager RecommendationEngine Database

Requirements	Student and Company recommendation [R12] The System shall use the Recommendation Engine to analyze student profiles to match them with internships proposed by companies [R13] The System shall suggest to Companies Students who match their requirements for an internship proposal [R14] The System shall suggest to Students Companies that offer internships aligned with their skills and experiences
Components	Webapp RecommendationEngine Database

Requirements	Company reach out [R15] The system shall allow Companies to review the suggested students to select the ones they wish to contact for an internship proposal [R16] The system shall notify Students selected by Companies for an internship
Components	Webapp InternshipManager Database

Requirements	Interview hosting [R17] The System shall allow Companies to propose three possible dates to Students for an interview [R18] The System shall allow Students to accept one of the proposed dates for an interview [R19] The System shall allow Students to ask Companies to provide new possible dates for an interview [R20] The System shall allow the hosting of online interviews between Companies and Students via a third party app
Components	Webapp InterviewManager Database

Requirements	Student admission [R21] The System shall allow companies to select students to admit to internships they interviewed for [R22] The System shall allow students to check for their admission to internships they interviewed for
Components	Webapp ApplicationManager InternshipManager Database

Requirements	Student feedback submission [R23] The System shall ask Students to provide feedback about an internship by filling out a form [R24] The System shall allow Companies to publicly comment on the feedback they received [R25] The System shall allow Students to see feedback about Companies from past internships they offered
Components	Webapp FeedbackManager InternshipManager Database

Requirements	Company feedback submission [R26] The System shall ask Companies to provide feedback about the Student's performance by filling out a form [R27] The System shall allow Students to publicly comment on the feedback they received [R28] The System shall allow Companies to see feedback about Students from past internships they partook in
Components	Webapp FeedbackManager StudentProfileManager Database

Chapter 5

Implementation, Integration and Test Plan

5.1 Overview

This section outlines the methodologies and strategies for implementing and testing the S&C platform. The primary goal of testing is to detect and address bugs introduced during development. The planned implementation strategies are detailed, alongside an explanation of the integration process for the various code components. These measures ensure the system meets the functional and non-functional requirements defined in the initial phases of the software development lifecycle.

5.2 Implementation plan

The **top-down approach** will be used to implement and integrate the various components of the system. This approach focuses on incremental integration and testing, starting from the highest-level interfaces and progressively working down to the lower-level modules.

The process begins with developing and testing top-level components, in this case, the Webapp containing the user interface and the API to communicate with the low-level components. Drivers are used to simulate interactions with these high-level interfaces, ensuring that they function correctly.

To facilitate testing during development, placeholder stubs are introduced for lower-level components that have yet to be implemented. These stubs simulate the behaviour of the missing components, enabling continuous testing of the integrated system without waiting for all components to be ready.

As individual components are completed, they replace their corresponding stubs, progressively expanding the system's functionality. This iterative process also involves introducing new stubs as deeper system levels are developed, ensuring that integration and testing can continue seamlessly.

Once all components have been implemented and integrated, the entire system undergoes comprehensive testing to validate its full functionality and adherence to the design specifications.

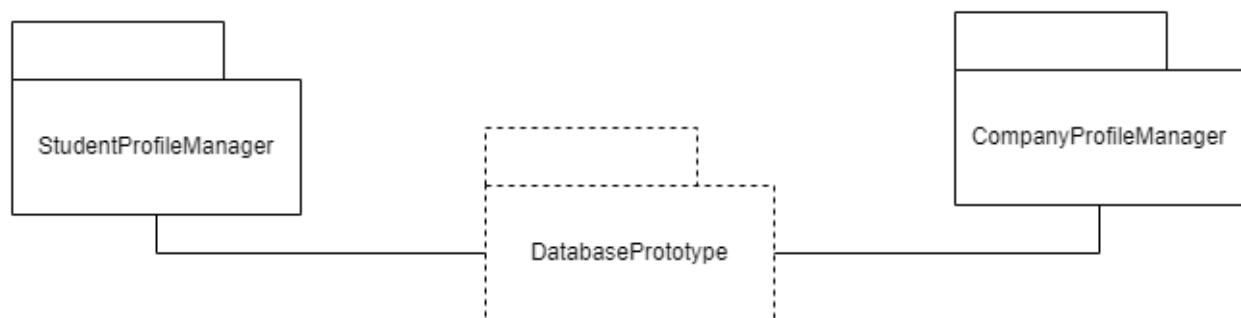
The advantages of this approach are: early validation through high-level interface testing, continuous incremental testing to address issues early and reduce risks, modular flexibility for parallel development, and controlled alignment with system design and requirements throughout the process.

5.3 Components Integration and Testing

As previously stated, the chosen approach for integration and testing consists of an incremental top-down approach. The order in which modules are implemented was chosen based on the top-down philosophy (higher-level modules first), with high consideration for the convenience and priority of each module. It is more logical to develop modules in close phases when they interact with each other or are present in the same use cases.

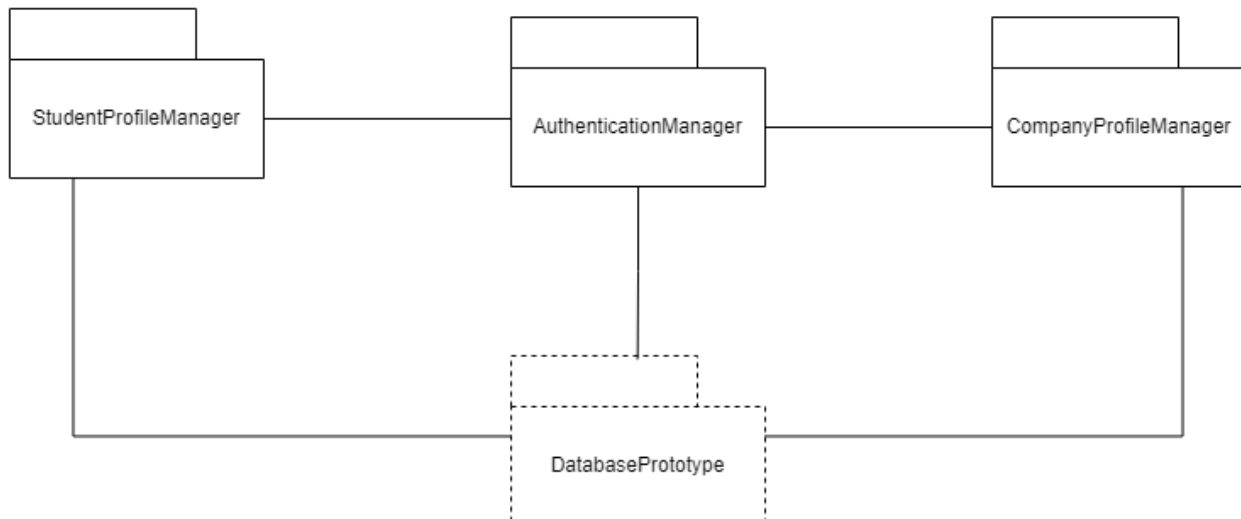
The Webapp component is the core controller and view of the system, responsible for managing user interactions and coordinating the behaviour of all other components. Given its critical role, it will be implemented first to establish a foundation for the integration and testing of other components. While it serves as the central hub, it will not be explicitly shown in the implementation diagrams to maintain diagram clarity and focus on the functional relationships among individual components.

Profile Managers



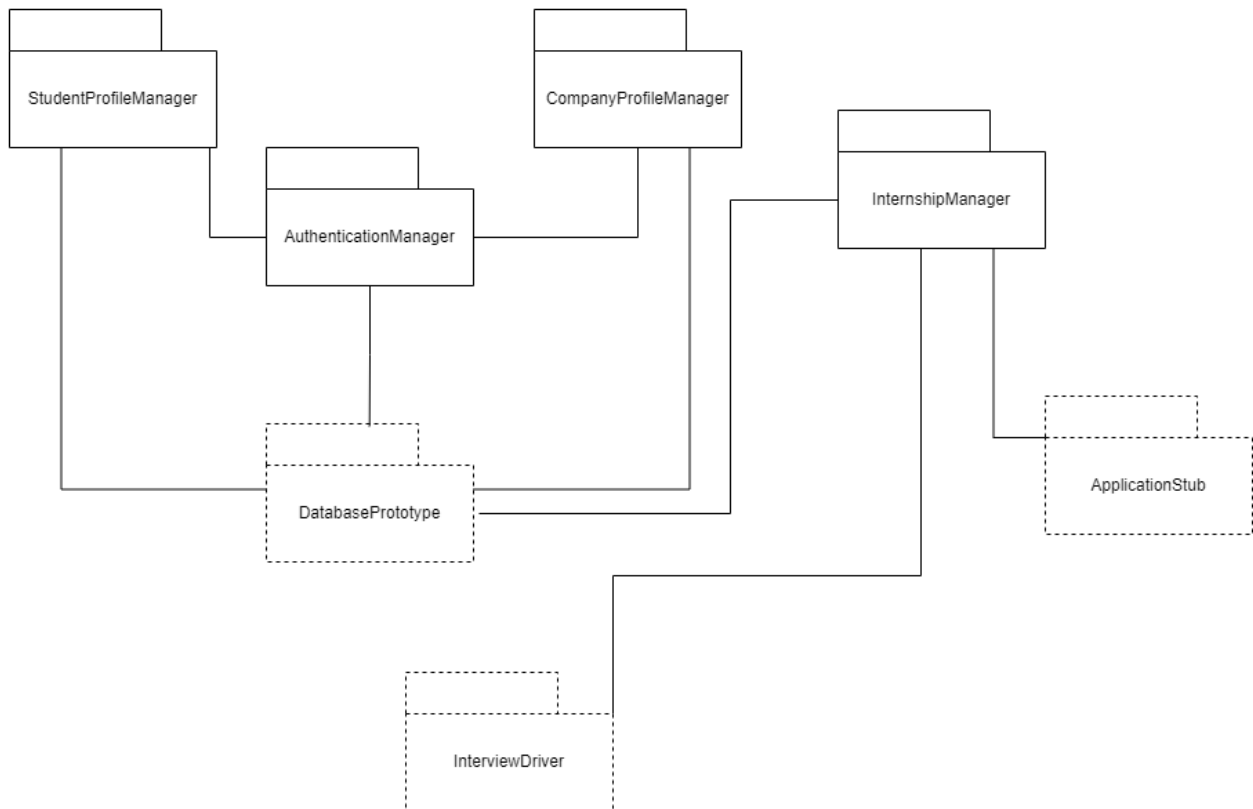
The first phase addresses the implementation of the **StudentProfileManager** and **CompanyProfileManager** modules. Both of these rely on the existence of persistent data; therefore, a database prototype or stub is expected to be used while a separate team develops the actual database. The stub could contain mock company profiles and mock student profiles, allowing testing of the main functionalities of the components as soon as they are released without waiting for the rest of the system.

Authentication



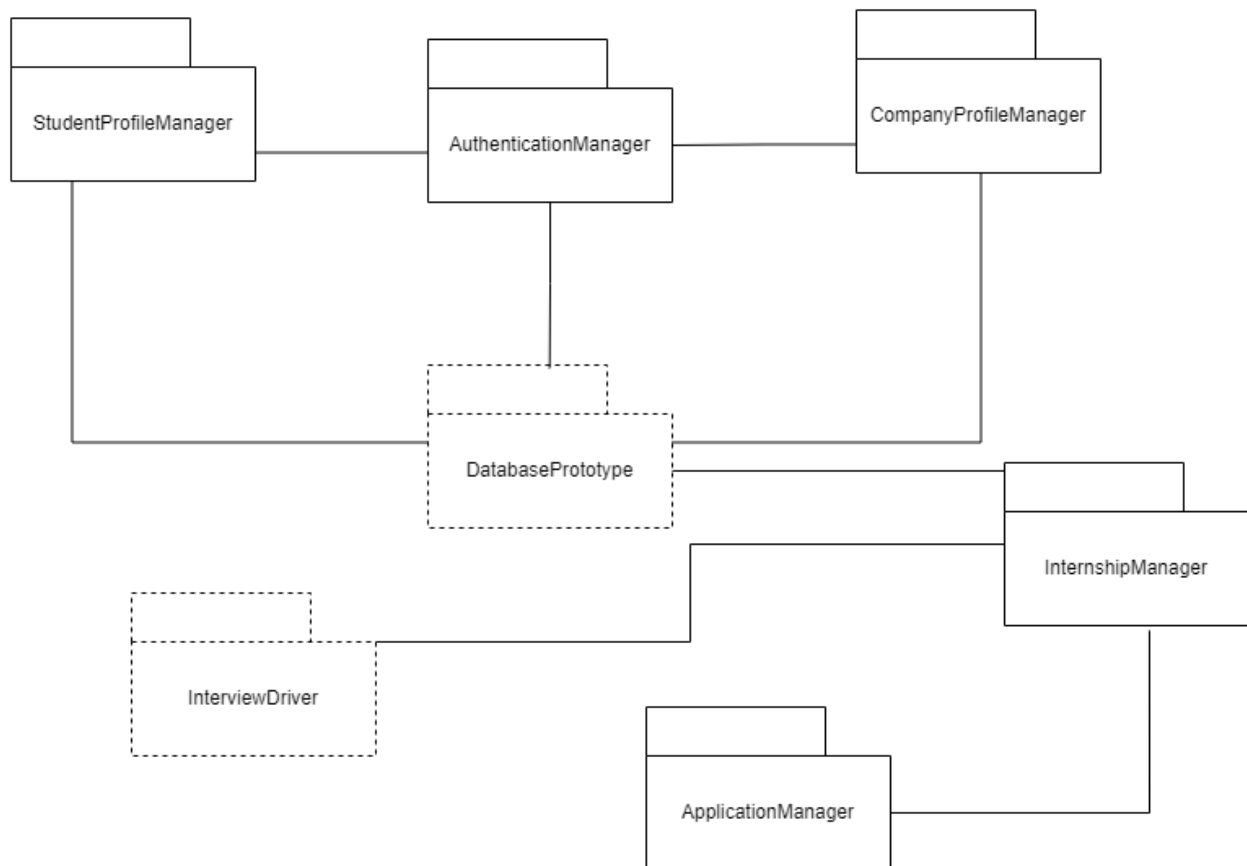
This phase focuses on the release of the **AuthenticationManager**, which is closely tied to the **StudentProfileManager** and the **CompanyProfileManager**. The only use cases in which this component is involved are the sign-up and login use cases, but they are crucial for a user-based platform such as S&C. The functionalities of the **AuthenticationManager** are also tied to the database since it will execute both read and write operations.

Internships

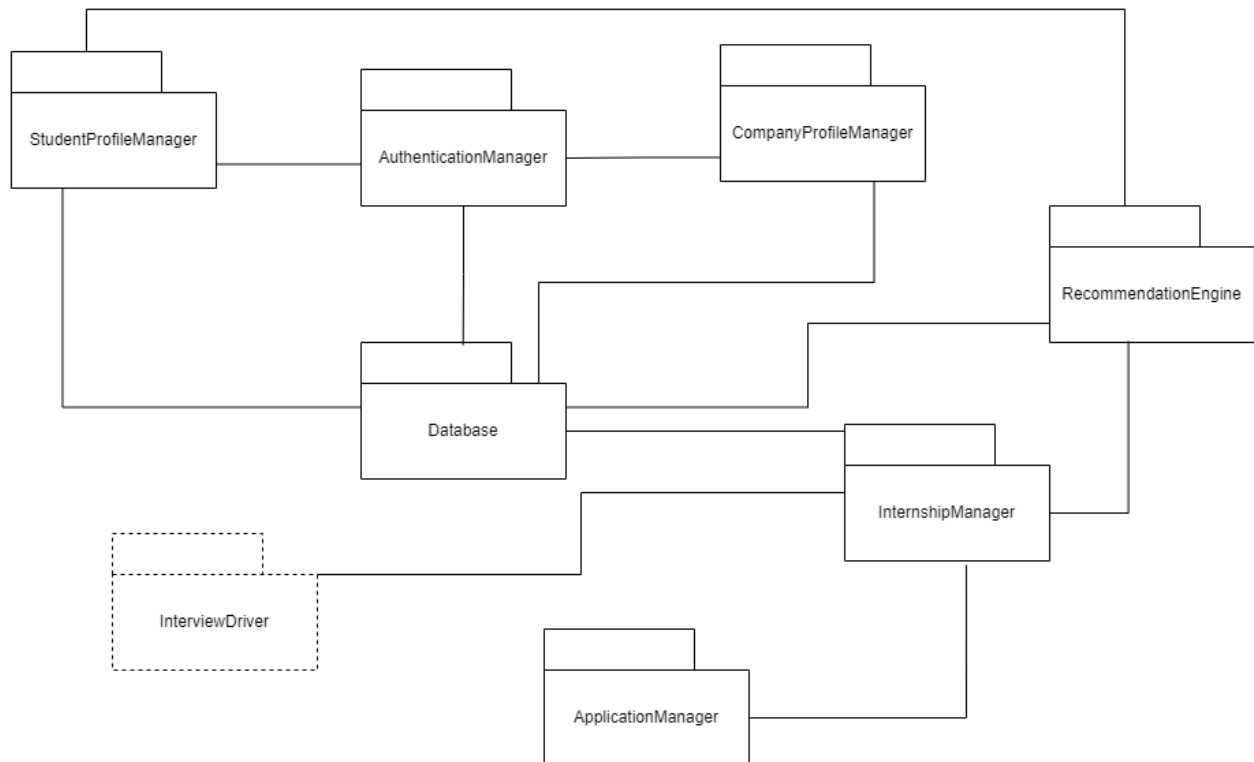


This phase focuses on the implementation of the heart of the system: the **InternshipManager**, which manages everything related to internships. This component plays a role in the majority of use cases and interacts with many other components. Therefore, it may be useful to include a stub for the **ApplicationManager** (to be used by the **InternshipManager**) and a driver for the **InterviewManager**, which relies on the **InternshipManager**.

Applications

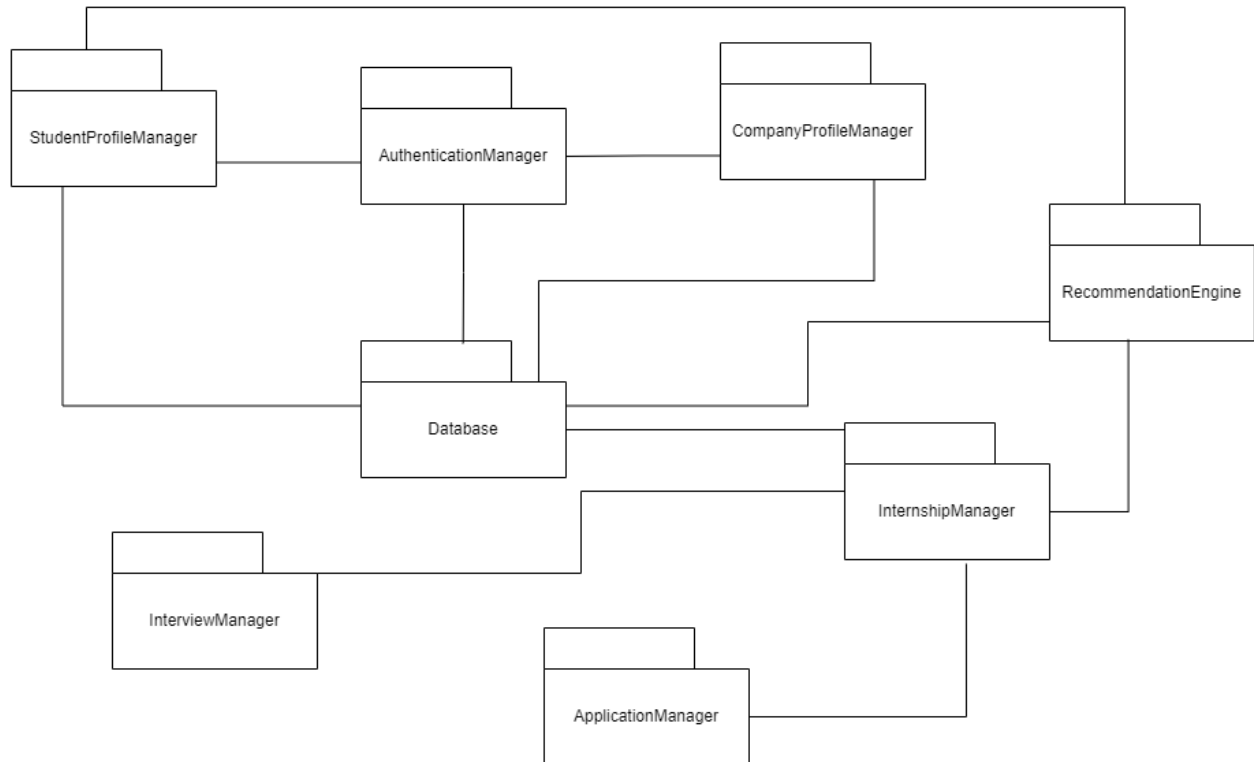


In this phase, the `ApplicationManager` is implemented. This is another crucial part of the system, closely tied to the `InternshipManager`. It will replace the stub previously used as a placeholder.

Recommendation Engine

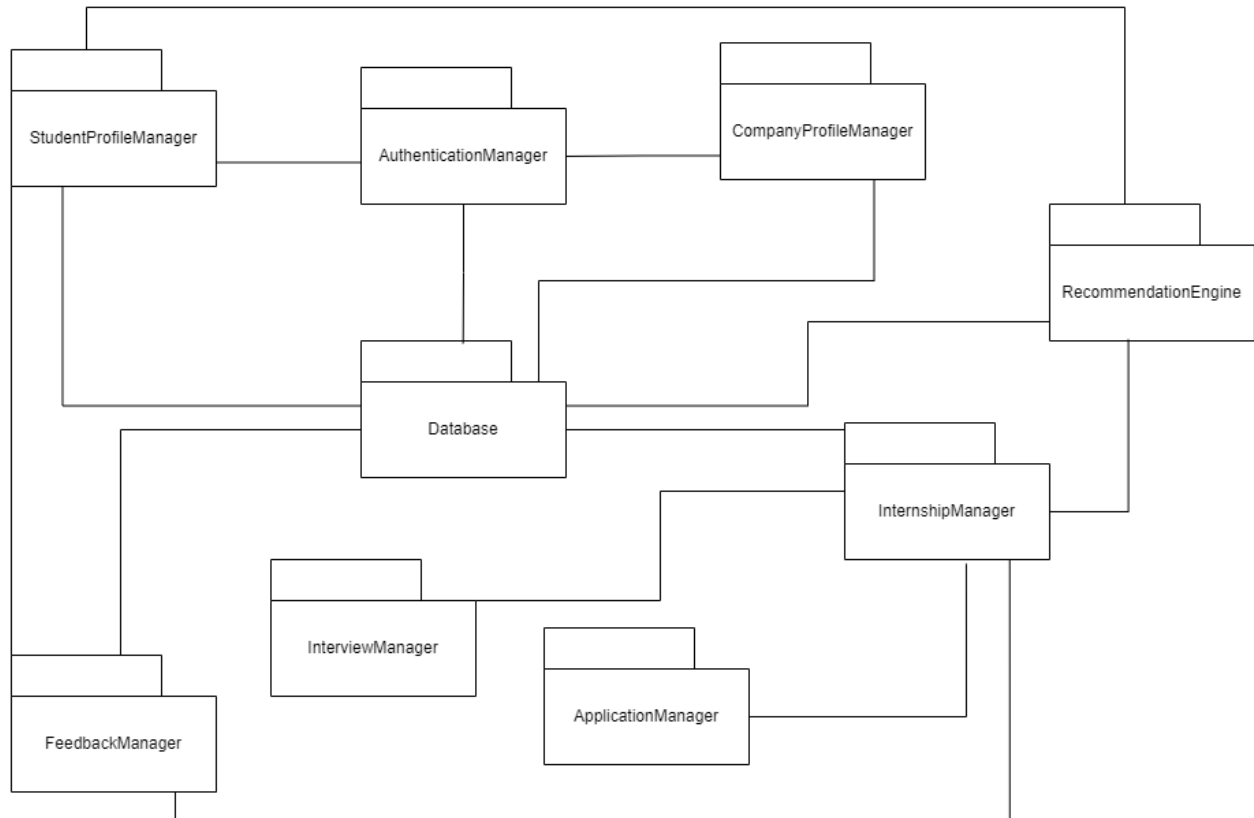
In this phase, the interface to the actual **RecommendationEngine** is built and subsequently unit-tested. The system interacts with the **RecommendationEngine** through an API. Since the **RecommendationEngine** is expected to be well-built, this phase should proceed smoothly. At this point in development, the database should be fully functional and properly interfaced with the software under construction.

Interviews



This phase implements the mechanisms for interviews through the **InterviewManager** component. This component must also interface with the third-party app for interviews. The actual component will replace the previously used driver.

Feedback and Comments



In the final phase, the **FeedbackManager** is developed. This component is left for last as it provides peripheral functionalities subordinate to other important and central parts of the software.

5.4 System testing

The S&C platform must be thoroughly tested to ensure its functionalities meet the expectations defined during the design and development phases. Each system component will undergo rigorous testing to identify and resolve potential issues. Some components may need stub and driver implementations to simulate interactions with missing or incomplete modules during early development stages: this allows us to test the components separately and independently from one another although they interact with each other. Once individual components are validated, they are incrementally integrated into the overall system for further testing.

The primary goal of system testing is to verify that the platform fulfils both functional and non-functional requirements as specified in the RASD document. This process requires collaboration among developers, testers, and stakeholders to ensure comprehensive coverage. The following testing phases will be carried out:

- **Functional Testing** This phase ensures that the platform's functionalities align with the requirements. For S&C, this involves:
 - testing whether students can upload their CVs, browse and apply for internships, and participate in interviews
 - verifying the recommendation engine's ability to suggest internships and candidates accurately
 - ensuring companies can create and manage internship announcements, review applications, and manage interviews effectively
 - ensuring that both companies and students can provide feedback on each other and respond to comments
- These tests will primarily follow the use cases outlined in the RASD document.
- **Performance Testing** The platform must handle peak loads, especially during high-traffic periods such as application deadlines. Performance testing will:
 - detect bottlenecks affecting response times, system throughput, and resource utilization
 - identify optimization opportunities for algorithms managing CV matching and recommendations
 - simulate workloads to ensure acceptable system performance under expected and peak conditions
- **Usability Testing** This phase focuses on evaluating the user interface and overall user experience. Usability tests will:
 - observe real users (students and companies) performing typical tasks
 - gather feedback on the ease of use and intuitiveness of the platform's design
 - make iterative improvements to address usability concerns
- **Load Testing** Load testing will expose the system to increasing workloads to determine its limits. For S&C, this includes:
 - simulating thousands of concurrent users uploading CVs or browsing internships
 - stressing the recommendation engine with large datasets of CVs and internship postings
 - ensuring the system remains responsive during prolonged periods of high activity
- **Stress Testing** Stress testing will ensure the platform recovers after failures by:
 - overloading system resources (e.g., memory, CPU) and observing recovery mechanisms
 - simulating unexpected network failures and verifying system stability
 - validating the integrity of stored data after failure scenarios

5.5 Additional Specifications on Testing

To ensure continuous improvement, the development team will collect feedback at every stage of testing. This feedback will guide adjustments to the platform before deployment.

Alpha Testing During alpha testing, stakeholders such as students and companies will interact with the platform in a controlled environment. Usability experts may design structured questionnaires to assess satisfaction levels and pinpoint usability issues. This phase will identify malfunctions and areas for improvement before proceeding to beta testing.

Beta Testing Beta testing involves releasing the platform to a limited group of real users under production-like conditions. Key objectives include:

- uncovering bugs or issues in a live environment
- collecting feedback on user satisfaction and system functionality
- logging system errors to be addressed before general release

Post-release Monitoring After deployment, the platform will continue to be monitored for issues. Logs will be automatically generated and sent to developers to facilitate debugging and future updates. Regular feedback from users will further ensure that the platform evolves to meet their needs.

Chapter 6

Effort Spent

This section contains a record of the effort spent by each group member expressed in hours of work for each section of each chapter

Chapter	Section	Ilardi's hours	Luraghi's hours	Collaborative hours
Chapter 1	Section 1.1, 1.2	0	0	1
	Sections 1.3, 1.4, 1.5, 1.6	0	0	0.5
Chapter 2	Section 2.1	0	0.5	0
	Section 2.2	4	3	3
	Section 2.3	0	0.5	0
	Section 2.4	6	4	2
	Section 2.5	0	3	3
	Section 2.6	0	1	0.5
	Section 2.7	0	0	0.5
Chapter 3		0	1	0
Chapter 4		0	1.5	0
Chapter 5	Section 5.1	0	0	0.5
	Section 5.2	0	1	0.5
	Section 5.3	3	0	0
	Section 5.4	1	0	0
	Section 5.5	0.5	0	1
Total		14.5	15.5	12.5